

ProteoRAG 开发记录

(一)

胡济梁

20260517

蛋白研究与蛋白质组学相关的 RAG 系统
与 AI 智能体

目 录

- 目 录.....2
- 第一章 大模型与 GenAI 综合.....9
 - (一) AI 大模型综合信息：9
 - 1. 什么是 AI 大模型？9
 - 2. AI 大模型的主要类型 错误!未定义书签。
 - 1) 按模型功能分类.....14
 - 2) 按应用领域分类.....错误!未定义书签。
 - 3. AI 大模型的主要用途 错误!未定义书签。
 - 1) 自然语言处理（NLP）错误!未定义书签。
 - 2) 计算机视觉（CV）错误!未定义书签。
 - 3) 专业领域应用.....错误!未定义书签。
 - 4) 企业与开发场景.....错误!未定义书签。
 - 5) 智能体与自动化.....错误!未定义书签。
 - 4. 大模型学习相关信息 错误!未定义书签。
 - 1) 大模型专家所需的数学基础？？（通义 2025）错误!未定义书签。
 - 2) 如何成为大模型开发与应用专家？（通义 2025）错误!未定义书签。
 - (二) 大语言模型综合信息： 错误!未定义书签。
 - 1. 什么是大语言模型？代表：ChatGPT..... 错误!未定义书签。
 - 2. 大语言模型和生成式 AI 存在交集 错误!未定义书签。
 - 3. 百度文心一言等大模型获批上线（2023.9.1） 错误!未定义书签。
 - (三) 生成式 AI 综合信息： 错误!未定义书签。
 - 1. 什么是生成式 AI? 错误!未定义书签。
 - 2. 如何成为 AI 绘图与视频生成专家？（通义 2025） 错误!未定义书签。

一、基础学习：打下理论与工具基础**	错误!未定义书签。
二、工具掌握：精通核心工具与 workflows**	错误!未定义书签。
三、进阶技能：模型开发与优化**	错误!未定义书签。
四、实战项目：积累作品与经验**	错误!未定义书签。
五、持续提升：跟踪前沿与行业动态**	错误!未定义书签。
六、硬件与环境配置建议**	错误!未定义书签。
七、总结路径图**	错误!未定义书签。
第二章 大模型与生成式 AI 常用平台	错误!未定义书签。
(一) Ollama 大模型平台：LLM 服务平台	错误!未定义书签。
1. Ollama 大模型平台简介	错误!未定义书签。
(二) Hugging Face：模型仓库；机器学习的 GitHub；... ..	错误!未定义书签。
1. Hugging Face 平台简介	错误!未定义书签。
2. 关于模型的不同版本与文件格式	错误!未定义书签。
3. Diffusers 库 (library)	错误!未定义书签。
1) Diffusers 库简介	错误!未定义书签。
2) 主要组件	错误!未定义书签。
(三) 国内镜像平台	错误!未定义书签。
1. 异型岛/互链高科	错误!未定义书签。
第二章 Transformer 模型	错误!未定义书签。
(一) Transformer 模型基本原理	错误!未定义书签。
0. Transformer explainer (以 GPT-2 small 模型为例)	错误!未定义书签。
1. 基本定义与组件	错误!未定义书签。
2. Embedding	错误!未定义书签。
3. Transformer Block	错误!未定义书签。

4. MLP: Multi-Layer Perceptron	错误!未定义书签。
5. Output Probabilities	错误!未定义书签。
6. Auxiliary Architectural Features	错误!未定义书签。
1) 层归一化 (Layer Normalization): 训练“稳定器”	错误!未定义书签。
2) 丢弃法 (Dropout): 防止“死记硬背”	错误!未定义书签。
3) 残差连接 (Residual Connections): 信息“快递专线”	错误!未定义书签。
第三章 大模型与 GenAI 商业模式	错误!未定义书签。
(一) 商业模式一: 云服务器部署	错误!未定义书签。
1. 项目介绍	错误!未定义书签。
第四章 Gemini	错误!未定义书签。
(一) Gemini 简介	错误!未定义书签。
(1)	错误!未定义书签。
(二) Gemini 应用: 数学函数图形 APP-	错误!未定义书签。
(1) 开发过程	错误!未定义书签。
(2) 本地运行	错误!未定义书签。
第四章 ChatGPT	错误!未定义书签。
(一) ChatGPT 综合信息	错误!未定义书签。
(一) ChatGPT 简介	错误!未定义书签。
(1)	错误!未定义书签。
(二) ChatGPT 相关资讯与进展	错误!未定义书签。
1. ChatGPT 与 Wolfram 结合!	错误!未定义书签。
2. 2023.11.7 OpenAI 发布 GPT-4 Turbo, 史上最强 ChatGPT 来了!	错误!未定义书签。

第五章 DeepSeek 错误!未定义书签。

(一) DeepSeek 综合信息: 错误!未定义书签。

1. 简介(V3/R1)..... 错误!未定义书签。

(二) DeepSeek R1 本地部署: 错误!未定义书签。

1. 部署方式一: ollama 平台 (B 站-Jack-Cui) 错误!未定义书签。

2. 部署方式一: ollama 平台 (自我部署-重医 HPC) .. 错误!未定义书签。

第六章 Stable Diffusion 错误!未定义书签。

(一) Stable Diffusion 综合信息: 错误!未定义书签。

1. Stable Diffusion 简介 错误!未定义书签。

2. Stability AI 公司简介 错误!未定义书签。

3. Stable Diffusion 不同版本的区别 错误!未定义书签。

(二) HPC 部署与使用(V1-5 版本): 错误!未定义书签。

1. HPC 平台-无 SUDO 权限- (不使用容器和 WEBUI) 错误!未定义书签。

1)在无 SUDO 权限的 HPC 平台上布置 Stable Diffusion? (不使用容器和 WEBUI)
.....错误!未定义书签。

1-2) 20250428 自己在重医 HPC 上部署 (失败)错误!未定义书签。

1-3) 20250503 自己在重医 HPC 上部署 (失败)错误!未定义书签。

1-4) 20250505 自己在重医 HPC 上部署 (成功)错误!未定义书签。

2. HPC 平台-无 SUDO 权限- (使用容器) 错误!未定义书签。

1) 使用 Singularity 容器错误!未定义书签。

(三) HPC 部署与使用(V3.5 版本): 错误!未定义书签。

1. Stable Diffusion 3.5 - 简介 错误!未定义书签。

2. Stable Diffusion 3.5 – 不同版本 错误!未定义书签。

1) Stable Diffusion 3.5 Large 介绍.....错误!未定义书签。

3. Stable Diffusion 3.5 – 使用方式	错误!未定义书签。
4. Stable Diffusion 3.5 – license	错误!未定义书签。
5. Stable Diffusion 3.5 Large – 安装与使用(总体介绍).	错误!未定义书签。
1) 总信息网址与 access 申请	错误!未定义书签。
2) 不同的安装源:	错误!未定义书签。
6. Stable Diffusion 3.5 Large – 安装与使用 (diffusers 途径)	错误!未定义书签。
1) 简介与文件下载.....	错误!未定义书签。
2) 20250512 自己在重医 HPC 上部署 (成功)	错误!未定义书签。
(四) AWS 部署与 WEB APP 开发:	错误!未定义书签。
1. 总体方案 (通义千问提供 20250520)	错误!未定义书签。
2. 20250520 自己 AWS 部署与 WEB APP 开发 ()	错误!未定义书签。
一、在 AWS 上部署 Stable Diffusion 模型	错误!未定义书签。
1. 准备 AWS 资源	错误!未定义书签。
2. 部署 Stable Diffusion 模型	错误!未定义书签。
(五) AWS 部署与移动端 APP 开发:	错误!未定义书签。
1. 总体方案 (通义千问提供 20250520)	错误!未定义书签。
第七章 其他大模型.....	错误!未定义书签。
(一) Llama 模型	错误!未定义书签。
1. Llama 简介:	错误!未定义书签。
(二) Gemma 模型 (在 Ollama 平台安装与部署)	错误!未定义书签。
1. Gemma 简介: 轻量级开源大模型	错误!未定义书签。
2. 在 Linux 上部署 (基于 Ollama)	错误!未定义书签。
3. 在 Windows 上部署 (基于 Ollama)	错误!未定义书签。



第一章 ProteoRAG-V1-开发

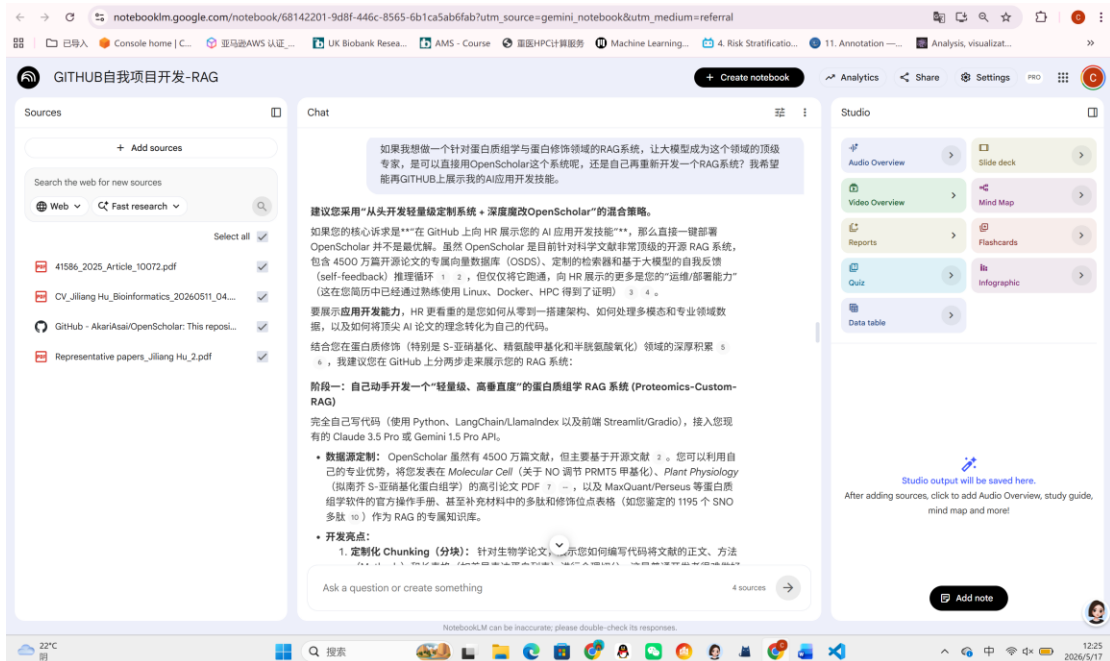
（一）项目综合信息：

1. 什么

语言大型（LLM, Large Language Models）

特点：

(二) Notebook LM 的项目建议



Sources

41586_2025_Article_10072.pdf

◆ Source guide

The provided text introduces **OpenScholar**, a pioneering, fully open **retrieval-augmented language model** specifically engineered to automate and enhance **scientific literature synthesis**. To address the common pitfalls of general AI—such as **hallucinated citations** and outdated information—the researchers developed a specialized pipeline that draws from a massive **data store of 45 million open-access papers**, using an iterative **self-feedback loop** to refine the accuracy and grounding of its responses. To rigorously test this system, the authors also debuted **ScholarQABench**, a first-of-its-kind multidisciplinary benchmark featuring nearly 3,000 expert-level queries across fields like biomedicine and physics. The study concludes that OpenScholar not only significantly **outperforms proprietary models** like GPT-4o in citation reliability but is often **preferred by human experts** due to its superior breadth and organizational clarity.

Retrieval-augmented language mo...

Scientific literature synth...

OpenSch...

ScholarQABe...

Citation accur...

Synthesizing scientific literature with retrieval-augmented language models

Akari Asai^{1,2}, Jacqueline He^{1,7}, Rulin Shao^{1,7}, Weijia Shi¹, Amanpreet Singh², Joseph Chee Chang², Kyle Lo², Luca Soldaini², Sergey Feldman², Mike D'Arcy², David Wadden², Matt Latzke², Jenna Sparks², Jena D. Hwang², Varsha Kishore^{1,2}, Minyang Tian³, Pan Ji⁴, Shengyan Liu³, Hao Tong³, Bohao Wu³, Yanyu Xiong⁵, Luke Zettlemoyer¹, Graham Neubig⁶, Daniel S. Weld^{1,2}, Doug Downey², Wen-tau Yih¹, Pang Wei Koh^{1,2} & Hannaneh Hajishirzi^{1,2} ✉

Scientific progress depends on the ability of researchers to synthesize the growing body of literature. Can large language models (LLMs) assist scientists in this task? Here we introduce OpenScholar, a specialized retrieval-augmented language model (LM) that answers scientific queries by identifying relevant passages from 45 million open-access papers and synthesizing citation-backed responses. To evaluate OpenScholar, we develop ScholarQABench, the first large-scale

1. Prompt:

如果我想做一个针对蛋白质组学与蛋白修饰领域的 RAG 系统，让大模型成为这个领域的顶级专家，是可以直接用 OpenScholar 这个系统呢，还是自己再重新开发一个 RAG 系统？我希望能再 GITHUB 上展示我的 AI 应用开发技能。

1) Response:

建议您采用“从头开发轻量级定制系统 + 深度魔改 OpenScholar”的混合策略。

如果您的核心诉求是***“在 GitHub 上向 HR 展示您的 AI 应用开发技能”***，那么直接一键部署 OpenScholar 并不是最优解。虽然 OpenScholar 是目前针对科学文献非常顶级的开源 RAG 系统，包含 4500 万篇开源论文的专属向量数据库 (OSDS)、定制的检索器和基于大模型自我反馈 (self-feedback) 推理循环，但仅仅将它跑通，向 HR 展示的更多是您的“运维/部署能力”（这在您简历中已经通过熟练使用 Linux、Docker、HPC 得到了证明）。

要展示应用开发能力，HR 更看重的是您如何从零到一搭建架构、如何处理多模态和专业领域数据，以及如何将顶尖 AI 论文的理念转化为自己的代码。

结合您在蛋白质修饰（特别是 S-亚硝基化、精氨酸甲基化和半胱氨酸氧化）领域的深厚积累，我建议您在 GitHub 上分两步走来展示您的 RAG 系统：

阶段一：自己动手开发一个“轻量级、高垂直度”的蛋白质组学 RAG 系统 (Proteomics-Custom-RAG)

完全自己写代码（使用 Python、LangChain/LlamaIndex 以及前端 Streamlit/Gradio），接入您现有的 Claude 3.5 Pro 或 Gemini 1.5 Pro API。

- **数据源定制：** OpenScholar 虽然有 4500 万篇文献，但主要基于开源文献。您可以利用自己的专业优势，将您发表在 *Molecular Cell*（关于 NO 调节 PRMT5 甲基化）、*Plant Physiology*（拟南芥 S-亚硝基化蛋白质组学）的高引论文 PDF，以及 MaxQuant/Perseus 等蛋白质组学软件的官方操作手册、甚至补充材料中的多肽和修饰位点表格（如您鉴定的 1195 个 SNO 多肽）作为 RAG 的专属知识库。

- **开发亮点：**

1. **定制化 Chunking (分块)：** 针对生物学论文，展示您如何编写代码将文献的正文、方法 (Methods) 和长表格 (如差异表达蛋白列表) 进行合理切分，这是普通开发者很难做好的。
2. **API 灵活调用：** 展示您能熟练使用大模型 API，并且能设计出系统架构图。

- **HR 吸睛点：** 向 HR 证明您具备了完整、独立的 AI Agent/RAG 软件工程开发与全栈构建能力。

阶段二：Fork 并“魔改” OpenScholar 系统 (OpenScholar-Proteomics-Edition)

当您有了一个基础的 RAG 后，您可以直接在 GitHub 上 Fork 官方开源的 OpenScholar 代码库 (<https://github.com/AkariAsai/OpenScholar>)，并在 README 中说明您是如何对其进行二次开发的。

- **引入 OpenScholar 的核心机制：** OpenScholar 最强大的地方在于它的自我反馈推理循环 (self-feedback inference loop)。生成器会先输出一个初稿和占位符引用，然后自动生成评估反馈 (例如：“答案仅包含了经验结果，请补充其他机制”)，并根据反馈发起二次检索，直到满足要求后再输出最终答案。
- **您的魔改开发：** 您可以修改它的源码，将其后端模型替换为您自己部署的本地 Ollama / vLLM 接口 (这也是 OpenScholar 官方原生支持的加速方案)。更重要的是，您可以将它的检索范围限定在特定的医学与生信数据库中，或者将您在阶段一中构建的“蛋白修饰专属数据库”注入到它的 OSDS 数据存储中。
- **专业能力降维打击：** 让系统专门回答如“在胰腺导管腺癌 (PDAC) 中，STAT3 的 S-亚硝基化如何影响细胞增殖？”这类高精尖问题。
- **HR 吸睛点：** 向 HR 证明您不仅能写基础代码，还能深入理解和二次开发基于《Nature》顶刊级别的复杂大模型推理管线 (custom inference

pipeline)，并且能够解决“长文本引用准确率（Citation accuracy）”和“降低幻觉（Hallucinations）”这些当前 AI 企业面临的核心痛点。

总结给您的行动建议： 在 GitHub 建立一个名为 Proteomics-RAG-Expert 的仓库。

1. **基础版 (v1.0):** 纯手写代码，基于 LangChain + Claude/Gemini API，让系统能精准回答您个人 CV 和代表作中的技术细节。
2. **进阶版 (v2.0):** 声明参考了 OpenScholar 的架构，引入交叉编码器重排（cross-encoder reranker）和自我反馈推理（self-feedback）代码逻辑，让系统能够像顶级专家一样，针对复杂的蛋白质组学与多组学问题撰写带有准确引用的长篇综述。

这样，您既展示了极强的代码重构与架构设计能力，又完美融合了您的顶尖生物医学背景，将成为您应聘 AI 医疗岗位的“杀手锏”。

(三) Gemini 提供的初步方案

1. Prompt

我是一名刚接触 RAG 的生信工作人员，目前直接上手 OpenScholar 感觉有难度。我还是选择自己先开发一个基础的 RAG 项目：自己动手开发一个“轻量级、高垂直度”的蛋白质组学 RAG 系统 (Proteomics-Custom-RAG)

完全自己写代码（使用 Python、LangChain/LlamaIndex 以及前端 Streamlit/Gradio），接入您现有的 Claude 3.5 Pro 或 Gemini 1.5 Pro API。

数据源定制：OpenScholar 虽然有 4500 万篇文献，但主要基于开源文献。您可以利用自己的专业优势，将您发表在 Molecular Cell（关于 NO 调节 PRMT5 甲基化）、Plant Physiology（拟南芥 S-亚硝基化蛋白组学）的高引论文 PDF，以及 MaxQuant/Perseus 等蛋白质组学软件的官方操作手册、甚至补充材料中的多肽和修饰位点表格（如您鉴定的 1195 个 SNO 多肽）作为 RAG 的专属知识库。

开发亮点：

定制化 Chunking（分块）：针对生物学论文，展示您如何编写代码将文献的正文、方法（Methods）和长表格（如差异表达蛋白列表）进行合理切分，这是普通开发者很难做好的。

API 灵活调用：展示您能熟练使用大模型 API，并且能设计出系统架构图。

HR 吸睛点：向 HR 证明您具备了完整、独立的 AI Agent/RAG 软件工程开发与全栈构建能力。请提供具体详细的步骤与代码。

1) Response

这是一个非常明智的决定。直接拆解和克隆机构级的庞然大物（如 OpenScholar）容易在环境配置和算力瓶颈上受挫，而**从零开始手写一个高垂直度的 RAG 系统，是彻底掌握其底层逻辑的最佳路径。**

将你深厚的蛋白质组学背景（Molecular Cell 机制研究、Plant Physiology 组学数据）与最新的 LLM 技术结合，开发出专属的 Proteomics-Custom-RAG，这不仅能解决实际科研需求，更是向 HR 证明你具备从数据清洗到 AI 架构设计全栈能力的顶级项目。

为了在你的本地开发环境（如 WSL2/Ubuntu）中流畅运行，我们将采用轻量级但企业级的技术栈：**LangChain + FAISS（本地向量库）+ HuggingFace BGE（生物学友好的本地 Embedding）+ Gemini 1.5 Pro API + Streamlit。**

以下是完整的系统构建指南与核心代码。

第一步：系统架构设计（HR 吸睛点）

在你的 GitHub README.md 中，首先需要展示这样清晰的系统架构图逻辑：

- 数据摄入层 (Data Ingestion):** 分流处理非结构化数据（文献 PDF、MaxQuant 手册）与半结构化数据（SNO 多肽 Excel/CSV 表格）。
 - 定制处理层 (Custom Chunking):** * 文献：基于正则和科学论文结构（Abstract, Methods, Results）进行语义切分。
 - 表格：行级序列化（Row-level Serialization），将生信数据转化为大模型易读的自然语言。
 - 向量检索层 (Vector Store):** 使用 bge-large-en-v1.5 将文本向量化，存入本地 FAISS 内存数据库。
 - 生成引擎层 (Generator):** 组装专业 Prompt，调用具备超长上下文能力的 Gemini 1.5 Pro API 进行推理。
 - 交互展示层 (UI):** 基于 Streamlit 搭建极简科研问答界面。
-

第二步：环境配置 (WSL2 / Ubuntu)

在你的终端中创建虚拟环境并安装依赖：

Bash

```
# 创建独立环境
```

```
conda create -n proteo-rag python=3.10
```

```
conda activate proteo-rag
```

```
# 安装核心依赖库
```

```
pip install langchain langchain-google-genai langchain-huggingface langchain-community
```

```
pip install faiss-cpu pypdf pandas streamlit sentence-transformers
```

设置你的 Google Gemini API 密钥作为环境变量：

Bash

```
export GOOGLE_API_KEY="你的_gemini_api_key_写在这里"
```

第三步：核心壁垒 —— 定制化 **Chunking** 代码

这是该项目含金量最高的部分。普通开发者只会按字数切分，这会彻底破坏蛋白质序列或修饰位点的上下文。

创建一个名为 `data_processor.py` 的文件：

Python

```
import pandas as pd
```

```
from langchain_community.document_loaders import PyPDFLoader
```

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
```

```
from langchain_core.documents import Document
```

```
def process_sno_peptides_table(csv_path):
```

```
    """
```

亮点 1：行级序列化 (Row-level Serialization)。

针对补充材料中的长表格（如 1195 个 SNO 多肽），将其转化为大模型更容易检索的结构化段落。

```
    """
```

```
    df = pd.read_csv(csv_path)
```

```
    documents = []
```

```
    # 假设你的 CSV 包含: Protein_ID, Gene, Peptide, SNO_Site, Fold_Change
```

```
    for index, row in df.iterrows():
```

```
        text_content = (
```

```
            f"In the Arabidopsis S-nitrosylation (SNO) proteomics dataset, "
```

```
            f"protein {row['Protein_ID']} (encoded by gene {row['Gene']}) was  
identified. "
```

```
            f"The SNO modification is located at site {row['SNO_Site']} within  
the peptide sequence '{row['Peptide']}'. "
```

```
            f"Following NO treatment, the fold change of this modification was  
{row['Fold_Change']}".
```

```
        )
```

```
        metadata = {
```

```
            "source": csv_path,
```

```

        "data_type": "Proteomics_Table",

        "protein": row['Protein_ID'],

        "modification": "SNO"

    }

    documents.append(Document(page_content=text_content,
                              metadata=metadata))

return documents

```

```
def process_biology_literature(pdf_path):
```

```
    """
```

亮点 2：语义感知切分。

针对 Molecular Cell 等高质量文献，使用定制的特殊分隔符，尽量保持 Methods 或 Results 独立段落的完整性。

```
    """
```

```
    loader = PyPDFLoader(pdf_path)
```

```
    pages = loader.load()
```

```
    # 优先按论文常见大段落和双换行符切分
```

```
    custom_splitter = RecursiveCharacterTextSplitter(
```

```
        chunk_size=1200,
```

```
        chunk_overlap=200,
```

```

        separators=["\n\nAbstract", "\n\nIntroduction", "\n\nResults",
"\n\nExperimental Procedures", "\n\n", "\n", " "]

    )

    docs = custom_splitter.split_documents(pages)

    for doc in docs:

        doc.metadata['data_type'] = "Literature_Text"

    return docs

```

第四步：构建 RAG 核心引擎 (API 调用与检索链)

创建一个名为 `rag_engine.py` 的文件，负责向量化和调度 Gemini API。

Python

```

import os

from langchain_huggingface import HuggingFaceEmbeddings

from langchain_community.vectorstores import FAISS

from langchain_google_genai import ChatGoogleGenerativeAI

from langchain.chains import create_retrieval_chain

from langchain.chains.combine_documents import create_stuff_documents_chain

from langchain_core.prompts import ChatPromptTemplate

from data_processor import process_sno_peptides_table, process_biology_literature

def build_vector_store():

```

```
print("Loading and chunking data...")

# 这里替换为你实际的文件路径

# sno_docs = process_sno_peptides_table("data/1195_SNO_peptides.csv")

# paper_docs = process_biology_literature("data/Molecular_Cell_PRMT5.pdf")

# manual_docs = process_biology_literature("data/MaxQuant_Manual.pdf")


# 模拟合并文档（请确保你有测试文件）

# all_docs = sno_docs + paper_docs + manual_docs

all_docs = [] # 请替换为实际加载的文档


print("Initializing local BGE embedding model...")

# BGE 模型对生物学词汇和英文支持良好，且可在本地 CPU 快速运行

embeddings = HuggingFaceEmbeddings(model_name="BAAI/bge-large-en-
v1.5")


print("Building FAISS vector store...")

if not all_docs:

    raise ValueError("No documents loaded. Please provide PDF or CSV
data.")

vectorstore = FAISS.from_documents(all_docs, embeddings)


# 将向量库保存到本地，避免每次重启重复计算

vectorstore.save_local("faiss_proteomics_index")
```

```
return vectorstore
```

```
def get_rag_chain():
```

```
    # 1. 加载本地向量库
```

```
    embeddings = HuggingFaceEmbeddings(model_name="BAAI/bge-large-en-  
v1.5")
```

```
    if not os.path.exists("faiss_proteomics_index"):
```

```
        vectorstore = build_vector_store()
```

```
    else:
```

```
        vectorstore = FAISS.load_local("faiss_proteomics_index", embeddings,  
allow_dangerous_deserialization=True)
```

```
    # 检索策略：提取最相关的 top 5 个数据块
```

```
    retriever = vectorstore.as_retriever(search_kwargs={"k": 5})
```

```
    # 2. 初始化大语言模型 (Gemini 1.5 Pro)
```

```
    llm = ChatGoogleGenerativeAI(
```

```
        model="gemini-1.5-pro",
```

```
        temperature=0.1, # 保持严谨性，降低幻觉
```

```
        max_tokens=1024
```

```
    )
```

```
    # 3. 设计专家级 Prompt
```

```

system_prompt = (
    "You are an elite bioinformatics and proteomics AI assistant. "
    "You help structural biologists and researchers analyze post-translational modifications (PTMs), "
    "specifically arginine methylation (PRMT5) and S-nitrosylation. "
    "Answer the user's questions strictly based on the following retrieved context, which includes "
    "published peer-reviewed literature and high-confidence MS/MS peptide datasets. "
    "If quoting peptide sequences or fold changes, ensure exact numerical accuracy. "
    "If the answer is not contained in the context, clearly state that you do not know.\n\n"
    "Context:\n{context}"
)

```

```

prompt_template = ChatPromptTemplate.from_messages([
    ("system", system_prompt),
    ("human", "{input}"),
])

```

4. 组装 RAG 链

```

question_answer_chain = create_stuff_documents_chain(llm, prompt_template)

rag_chain = create_retrieval_chain(retriever, question_answer_chain)

```

```
return rag_chain
```

第五步：全栈构建 —— Streamlit 前端交互

创建一个名为 `app.py` 的文件，这不仅是给 HR 看的门面，也能方便你日常科研使用。

Python

```
import streamlit as st
```

```
from rag_engine import get_rag_chain
```

```
# 设置页面配置
```

```
st.set_page_config(page_title="Proteomics Custom RAG", page_icon="🧬",  
layout="wide")
```

```
st.title("🧬 Proteomics & PTM Custom RAG System")
```

```
st.markdown("""
```

```
**Expert System for Post-Translational Modifications** *Powered by Custom  
Chunking, BGE-Embeddings, and Gemini 1.5 Pro API.* This agent is grounded on  
high-impact literature (Molecular Cell, Plant Physiology) and proprietary MS/MS  
peptide datasets.
```

```
""")
```

```
# 初始化 RAG 引擎并存入 session_state 以免重复加载
```

```
if "rag_chain" not in st.session_state:

    with st.spinner("Initializing AI Agent and loading Vector Database..."):

        try:

            st.session_state.rag_chain = get_rag_chain()

            st.success("System Ready!")

        except Exception as e:

            st.error(f"Initialization failed (Please ensure data files exist and API key is set): {e}")

# 对话历史记录

if "messages" not in st.session_state:

    st.session_state.messages = []

# 显示历史对话

for msg in st.session_state.messages:

    with st.chat_message(msg["role"]):

        st.markdown(msg["content"])

# 用户输入区域

if prompt := st.chat_input("Ask about PRMT5 mechanism, MaxQuant settings, or specific SNO peptides..."):

    # 显示用户问题

    st.session_state.messages.append({"role": "user", "content": prompt})
```

```
with st.chat_message("user"):

    st.markdown(prompt)

# 生成并显示 AI 回答

with st.chat_message("assistant"):

    with st.spinner("Searching proprietary databases and literature..."):

        if "rag_chain" in st.session_state:

            response = st.session_state.rag_chain.invoke({"input": prompt})

            answer = response["answer"]

            st.markdown(answer)

# 可选：展示引用来源以增加可信度

with st.expander("View Retrieved Sources"):

    for i, doc in enumerate(response["context"]):

        source = doc.metadata.get('source', 'Unknown source')

        st.write(f"**Source {i+1}**": {source}")

        st.caption(f"{doc.page_content[:200]}...")

    st.session_state.messages.append({"role": "assistant", "content":
answer})
```

第六步：启动与测试

将准备好的测试数据（如你的论文 PDF 和一部分多肽 CSV）放到项目目录的 data/ 文件夹下，然后通过终端运行：

Bash

```
streamlit run app.py
```

浏览器会自动弹出一个极其专业的生物信息学问答界面。你可以尝试询问：

“Which peptide sequences contain an SNO modification and what are their fold changes?” 或者 *“According to the loaded literature, how does NO regulate PRMT5 methylation?”*。

这个项目的核心价值在于，它向所有人证明了你不只是会调用 API，而是懂得如何针对特定科学领域的数据结构进行底层的数据清洗和逻辑重构。随着后续对底层性能的追求，你还可以将向量检索库平滑替换为 C++ 或 Rust 驱动的更高性能组件。

1. Prompt:

前面这个基础项目是调用大模型吗？可以在我的 Thinkbook 16+ 上跑通吗？

1) Response

是的，这个基础项目调用了大语言模型，并且绝对可以在你的 Thinkbook 16+ 上流畅跑通。

这个项目之所以能在个人笔记本上运行，是因为它巧妙地采用了“算力分离”的架构。它不是把整个庞然大物塞进你的电脑，而是把工作拆分成了本地和云端两部分：

1. 最重负荷的“生成与推理”—— 交给云端 (API 调用)

在代码中，`ChatGoogleGenerativeAI(model="gemini-1.5-pro")` 这一行就是大模型的核心。

- **它在哪里跑？** 它并不消耗你电脑的 CPU 或显卡，而是通过网络发送请求，在 Google 的超级服务器集群上运行，然后把生成的文字结果传回你的电脑。
- **优势：** 经历了之前的硬件波动和彻底的系统重置，现在的首要任务是避免让电脑承受极端的高温和计算负载。强行在本地运行像 OpenScholar 那样的 8B 模型很容易诱发新的蓝屏或系统崩溃。而调用 API 就好比你在用浏览器看高清视频一样，对硬件毫无压力。

2. 中等负荷的“文本向量化 (Embedding)”—— 在本地跑

在代码中，`HuggingFaceEmbeddings(model_name="BAAI/bge-large-en-v1.5")` 负责把你喂给它的 PDF 和 Excel 表格变成数学向量。

- **它在哪里跑？** 在你的 Thinkbook 16+ 本地运行（通常在 CPU 或者集显上）。
- **优势：** 这个 BGE 模型是一个只有 1GB 左右的微型模型，你的电脑内存（哪怕只有 16GB）处理它绰绰有余。本地处理的好处是，你那些未

发表的机密数据（比如 SNO 多肽表格）在变成毫无意义的数字向量之前，完全不会泄露到互联网上，保证了科研数据的绝对安全。

3. 轻负荷的“UI 与逻辑流”—— 在本地跑

- Streamlit 的网页渲染、文档的定制化切分（Chunking）代码，都像普通的 Python 脚本一样在你的 WSL2/Ubuntu 环境中运行。占用资源极低，极其轻量。

总结

你相当于在 Thinkbook 16+ 上搭建了一个“指挥中心”。数据清洗、切分和建立本地私密索引都在你的笔记本上完成，而最耗费脑力的“阅读理解与回答”则通过网络外包给了当前地球上顶级的商业大模型。

你只需要确保两件事即可在电脑上完美跑通：

1. 本地 Python 环境（如 Conda）安装好了所需的几个基础库。
2. 电脑能够正常联网并访问相应的 API 服务。

1. Prompt:

我目前有 GOOGLE AI PRO 和 CLAUDE PRO 的订阅，请问需要将调用的模型改为更为合适的模型吗？

1) Response

这是一个“富裕的烦恼”，因为你目前手握的是当今地表最强的两个大模型生态。对于蛋白质组学 RAG 这个高度专业的垂直应用，选择哪个模型主要取决于你的核心诉求以及处理长表格的策略。

在给出建议前，需要先厘清一个现实的工程限制：**你的网页版订阅（每月 20 美元的 Google AI Pro / Claude Pro）并不等同于 API 额度。**

- **Google Gemini 1.5 Pro:** 在 Google AI Studio 中提供非常慷慨的**免费 API 额度**（每天 50 次超长上下文请求），非常适合个人开发者白嫖测试。
- **Claude 3.5 Sonnet:** Anthropic 的 API 需要额外绑卡充值（Pay-as-you-go），网页版订阅的额度不能用于代码 API 调用。

明确了这一点后，以下是针对你的生信 RAG 项目的深度对比与选择建议：

方案 A：继续使用 Gemini 1.5 Pro (擅长“暴力美学”与超长上下文)

如果你的补充材料表格特别巨大（比如上万行的鉴定肽段 CSV），或者你不想花太多精力写复杂的 Chunking 代码，Gemini 是首选。

- **核心优势：** 高达 200 万 token 的上下文窗口。你可以把整本《MaxQuant 官方手册》加上几十个大型组学表格作为 Context 一次性塞给它。
- **生信表现：** 能够在大海捞针式的检索中找到特定的 SNO 修饰位点，且多模态能力极强（未来可以直接喂给它包含 Western Blot 图像的 PDF）。
- **成本极佳：** 利用 AI Studio 的免费层，你的开发成本几乎为零。

方案 B: 切换至 Claude 3.5 Sonnet (擅长“极其严谨的科学推理”)

目前在学术界和开发者群体中，Claude 3.5 Sonnet 被公认为是处理复杂科学文本、编写代码以及遵循严格格式（如避免幻觉、必须带引用）的最佳模型。

- **核心优势：** 极高的逻辑推理能力和“老学究”般的严谨度。当你问它“NO 如何调节 PRMT5 甲基化”时，Claude 更擅长将多篇文献的机制进行完美的逻辑串联，并且语言表达比 Gemini 更接近人类顶级科学家的写作风格（Nature/Cell 级别的行文）。
- **开发辅助：** 在你写 RAG 系统的 Python 代码和排查 Bug 时，用你的网页版 Claude Pro 当编程助手是目前的最佳实践。
- **API 接入代码（如果你决定充值使用 Claude API）：**

你只需要把刚才 rag_engine.py 里的模型初始化代码替换成 LangChain 的 Anthropic 模块即可：

Python

```
# 需要先安装: pip install langchain-anthropic
```

```
import os
```

```
from langchain_anthropic import ChatAnthropic
```

```
# 设置环境变量
```

```
# export ANTHROPIC_API_KEY="your_api_key_here"
```

```
# 替换大模型初始化部分
```

```
llm = ChatAnthropic(
```

```
    model="claude-3-5-sonnet-20240620",
```

```
    temperature=0.1,
```

max_tokens=1024

)

终极建议：双剑合璧的开发流

不要局限于在代码里只用一个。既然你有双重 Pro 订阅，最完美的开发工作流程是这样的：

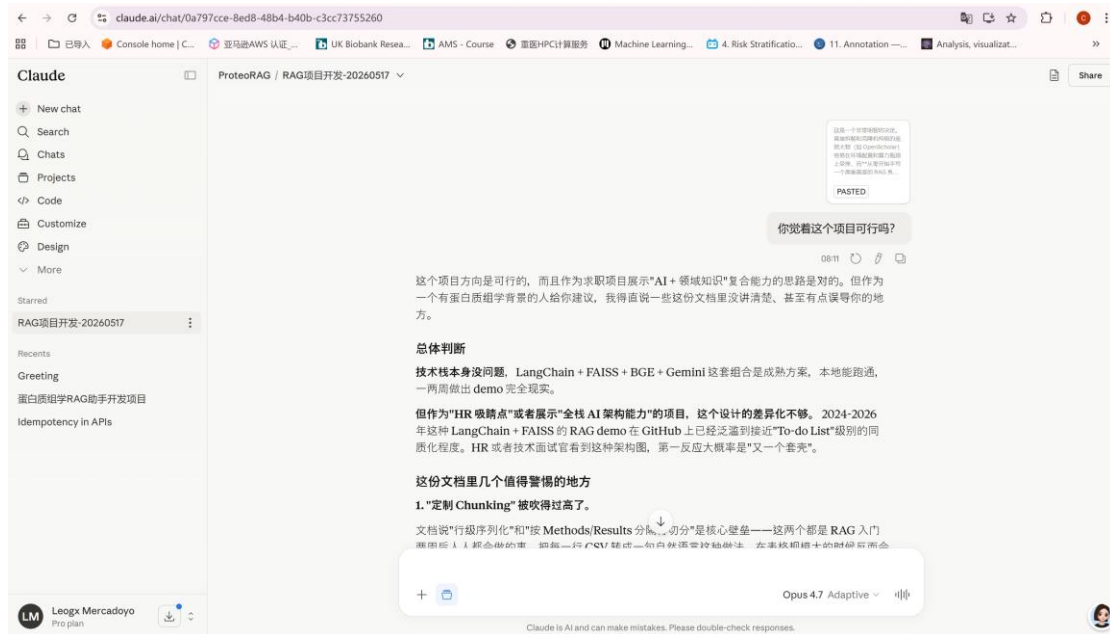
1. **让 Claude 写代码：** 在网页端打开 Claude Pro，把刚才我给你的系统架构和 Python 需求发给它，让它帮你细化 Chunking 的代码，甚至帮你写 Streamlit 的精美前端布局。
2. **用 Gemini 跑业务：** 在你的 Proteomics-Custom-RAG 代码中，通过 Google AI Studio 的免费 API 接入 Gemini 1.5 Pro，利用它超大的上下文窗口来“吞噬”你的高引文献和巨大的 SNO 多肽表格。

你的数据集里，除了文献和表格，是否包含结构化的基因注释（FASTA）或者 PDB 结构文件？如果有，我们在切分和嵌入（Embedding）阶段可能还需要引入生物信息学专属的工具。

（四）Claude 修改的详细方案

1. 你觉着这个项目可行吗？

Prompt:



你觉着这个项目可行吗？

Response:

这个项目方向是可行的，而且作为求职项目展示"AI + 领域知识"复合能力的思路是对的。但作为一个有蛋白质组学背景的人给你建议，我得直说一些这份文档里没讲清楚、甚至有点误导你的地方。

总体判断

技术栈本身没问题，LangChain + FAISS + BGE + Gemini 这套组合是成熟方案，本地能跑通，一两周做出 demo 完全现实。

但作为"HR 吸睛点"或者展示"全栈 AI 架构能力"的项目，这个设计的差异化不够。2024-2026 年这种 LangChain + FAISS 的 RAG demo 在 GitHub 上已经泛滥到接近"To-do List"级别同质化程度。HR 或者技术面试官看到这种架构图，第一反应大概率是"又一个套壳"。

这份文档里几个值得警惕的地方

1. "定制 Chunking" 被吹得过高了。

文档说"行级序列化"和"按 Methods/Results 分隔符切分"是核心壁垒——这两个都是 RAG 入门两周后人人都会做的事。把每一行 CSV 转成一句自然语言这种做法，在表格规模大的时候反而会造成检索灾难（1195 条 SNO 多肽变成 1195 个高度相似的向量，BGE 区分不出来，top-5 检索基本会乱套）。真正的表格 RAG 现在主流做法是 Text-to-SQL 或者结构化检索 + LLM 综合，不是行级序列化。

2. BGE 对生物学词汇"友好"这个说法不准确。

bge-large-en-v1.5 是通用英文 embedding，对蛋白名、修饰位点、肽段序列的语义表示其实很一般。蛋白质组学这种领域，你要么用 BioBERT / SciBERT / PubMedBERT 系列的领域 embedding，要么微调一个，否则"领域 RAG"的领域性只体现在数据上，不体现在检索上。

3. Gemini 1.5 Pro 在 2026 年的当下已经不是最优选择。 Gemini 2.5、Claude Opus 4.x、GPT-5 这一代模型在科学推理上都明显更强。如果是为了求职 demo，模型选择最好跟上。

4. 评估部分完全缺失。 一个能让 HR 眼前一亮的 RAG 项目，最关键的不是 UI 漂亮，而是有没有量化的 retrieval / generation 评估——recall@k、MRR、faithfulness、hallucination rate。没有 eval 的 RAG 项目在 2026 年招聘市场上几乎没有说服力，因为它无法区分"能跑"和"能用"。

如果是我，会怎么调整

如果你的目标是用这个项目找 AI/生信交叉岗位的工作，我建议把重心从"全栈 demo"挪到"领域深度 + 工程严谨度"，具体可以考虑：

- 换一个真问题做切入点。 比如"给定一个未知功能的 SNO 修饰位点，结合文献+数据库预测它的生物学意义"，这是一个文献里没有标准答案、需要 RAG 真正发挥作用的场景。

- 加入领域 embedding 的对比实验。BGE vs PubMedBERT vs 微调后的 BGE, 给出 retrieval 指标对比——这一项就足以让你和 99% 的 GitHub RAG 项目区分开。
- 加一个评估集和评估脚本。哪怕只有 50 条手工标注的 QA 对, 配上 RAGAS 或自定义 metric, 价值远大于 Streamlit UI。
- 诚实地写 limitations。在 README 里写清楚"行级序列化在 N>1000 时检索质量下降, 因此我引入了 X 策略"——这种自我批判的工程叙事, 比"企业级架构"这种话术对面试官有用得多。
- 考虑加 agentic 能力。比如让 LLM 自己决定调用文献检索还是查 UniProt API 还是查本地表格, 这才是 2026 年 RAG 的演进方向。

一句话总结

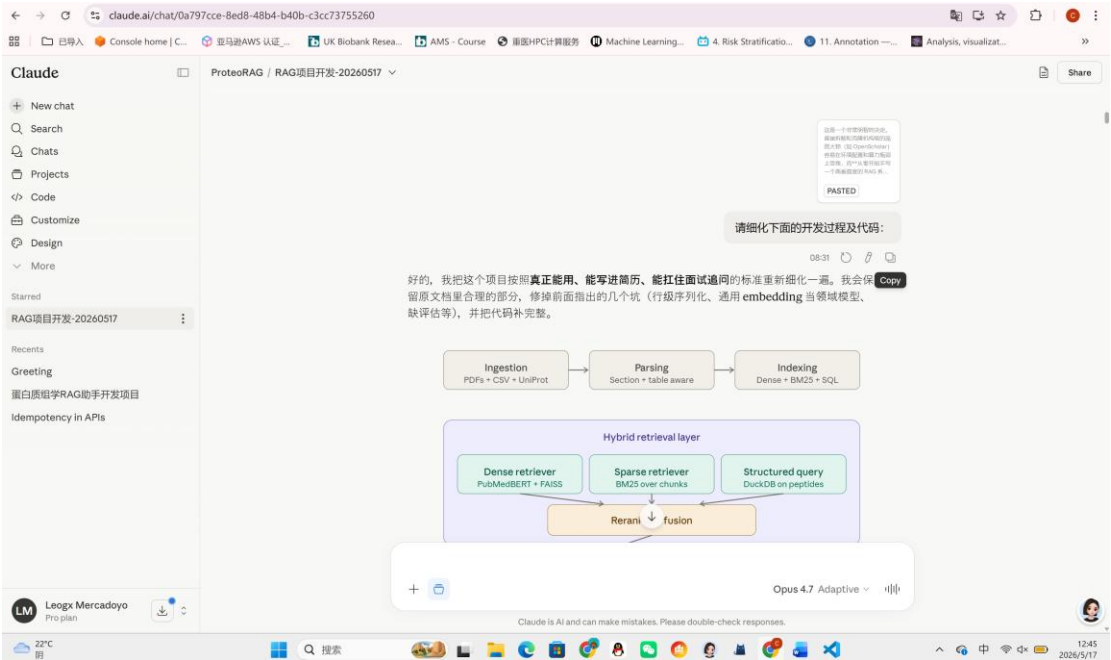
文档里的代码可以作为起点, 一周内跑通没问题。但不要把它当成最终交付物, 更不要相信"行级序列化 + 按章节切分 = 核心壁垒"这种话——那是 2023 年的卖点。

你愿意的话, 可以告诉我你具体想用这个项目解决什么科研问题 (或者瞄准什么类型的岗位), 我帮你重新设计一个差异化更强、技术深度更对得起你蛋白组学背景的方案。

2. 请细化下面的开发过程及代码

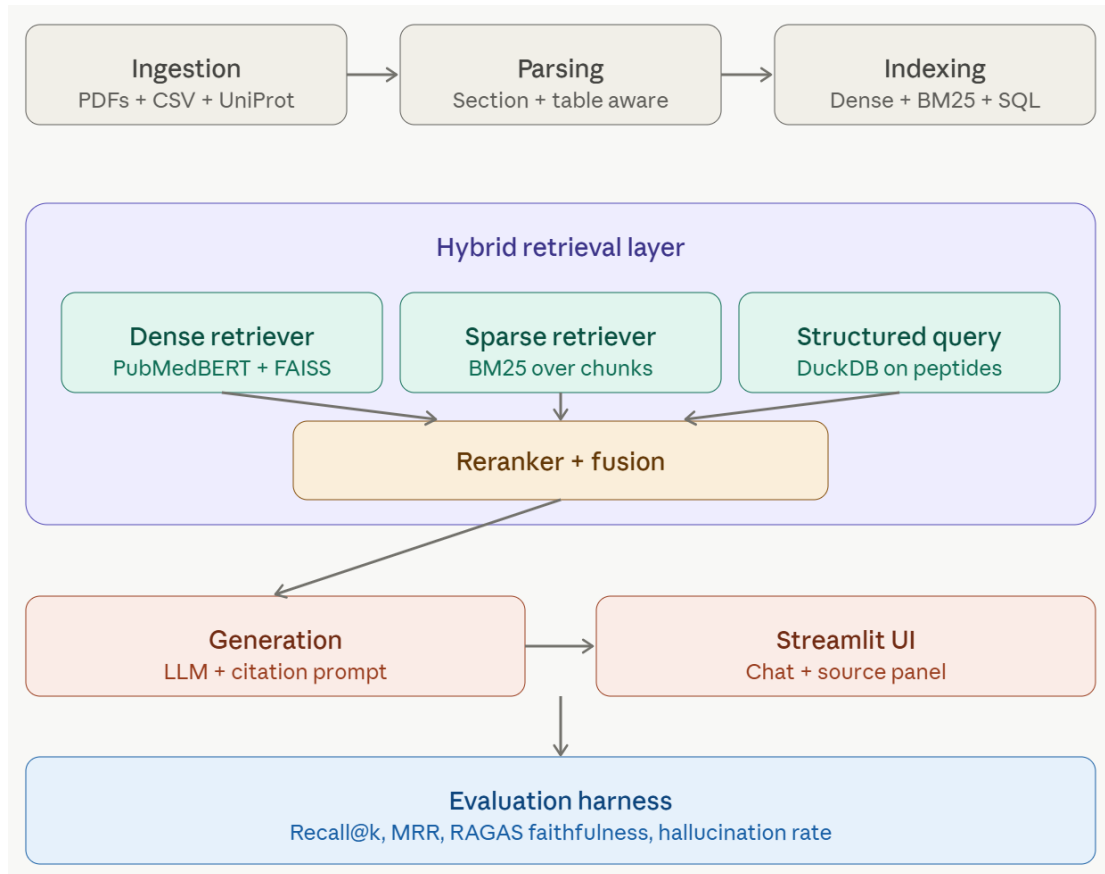
Prompt:

请细化下面的开发过程及代码：



Response:

好的，我把这个项目按照真正能用、能写进简历、能扛住面试追问的标准重新细化一遍。我会保留原文档里合理的部分，修掉前面指出的几个坑（行级序列化、通用 embedding 当领域模型、缺评估等），并把代码补完整。下面是按这个架构展开的完整开发流程和代码。



一、项目目录结构

先定好目录，方便后面所有代码都有明确归属：

proteomics-rag/

├── data/

│ ├── raw/ # 原始 PDF、CSV

│ ├── processed/ # chunk 后的 JSONL

│ └── eval/ # 评估集 QA pairs

├── src/

│ ├── __init__.py

│ ├── config.py # 路径、模型名、超参集中管理

│ └── parsers/

- | | └─ pdf_parser.py # 论文解析(GROBID 或 unstructured)
- | | └─ table_parser.py # 表格 → 结构化 + 自然语言双轨
- | └─ indexing/
- | | └─ chunker.py # 章节感知 chunking
- | | └─ dense_index.py # FAISS + 领域 embedding
- | | └─ sparse_index.py # BM25
- | | └─ sql_index.py # DuckDB 表格索引
- | └─ retrieval/
- | | └─ hybrid.py # RRF 融合
- | | └─ reranker.py # BGE-reranker 交叉编码器
- | └─ generation/
- | | └─ llm.py # LLM 客户端封装(支持多家)
- | | └─ prompts.py # 系统 prompt + 引用格式
- | └─ agent/
- | | └─ router.py # 决定走结构化查询还是语义检索
- | └─ eval/
- | | └─ build_eval_set.py # 半自动构造评估集
- | | └─ retrieval_metrics.py # Recall@k, MRR, nDCG
- | | └─ ragas_runner.py # faithfulness / answer relevance
- | └─ app.py # Streamlit 前端
- | └─ scripts/
- | └─ 01_ingest.py

```
|   |—— 02_build_index.py
|   |—— 03_run_eval.py
|—— tests/
|—— requirements.txt
|—— README.md
|—— .env.example
```

这种结构本身在面试里就是一个加分项——它说明你想过工程边界，而不是把所有东西塞进一个 notebook。

二、环境配置（修正版）

```
conda create -n proteo-rag python=3.11 -y
```

```
conda activate proteo-rag
```

```
# 核心
```

```
pip install langchain langchain-community langchain-core
```

```
pip install faiss-cpu rank-bm25 duckdb
```

```
# 解析
```

```
pip install pypdf pdfplumber unstructured[pdf] pandas
```

```
# Embedding 与 rerank（注意：用领域模型，不用通用 BGE）
```

```
pip install sentence-transformers transformers torch --index-url
```

```
https://download.pytorch.org/whl/cpu
```

LLM 客户端（多家备选）

```
pip install anthropic google-generativeai openai
```

评估

```
pip install ragas datasets
```

UI 与工程

```
pip install streamlit python-dotenv pydantic-settings tqdm
```

.env.example:

ANTHROPIC_API_KEY=

GOOGLE_API_KEY=

OPENAI_API_KEY=

LLM_PROVIDER=anthropic # anthropic | google | openai

LLM_MODEL=claude-opus-4-7

EMBEDDING_MODEL=pritamdeka/S-PubMedBert-MS-MARCO

RERANKER_MODEL=BAAI/bge-reranker-v2-m3

DATA_DIR=./data

三、配置模块（src/config.py）

集中管理所有可调参数，方便做消融实验：

```
from pathlib import Path
```

```
from pydantic_settings import BaseSettings, SettingsConfigDict
```

```
class Settings(BaseSettings):

    model_config = SettingsConfigDict(env_file=".env", extra="ignore")

    # 路径

    data_dir: Path = Path("./data")

    raw_dir: Path = Path("./data/raw")

    processed_dir: Path = Path("./data/processed")

    index_dir: Path = Path("./data/index")

    eval_dir: Path = Path("./data/eval")


    # LLM

    llm_provider: str = "anthropic"

    llm_model: str = "claude-opus-4-7"

    anthropic_api_key: str = ""

    google_api_key: str = ""

    openai_api_key: str = ""


    # Embedding & rerank

    embedding_model: str = "pritamdeka/S-PubMedBert-MS-MARCO"

    reranker_model: str = "BAAI/bge-reranker-v2-m3"
```

```

# Retrieval

dense_top_k: int = 20

sparse_top_k: int = 20

rerank_top_k: int = 5

rrf_k: int = 60


# Chunking

chunk_size: int = 800

chunk_overlap: int = 150


settings = Settings()

for d in [settings.processed_dir, settings.index_dir, settings.eval_dir]:

    d.mkdir(parents=True, exist_ok=True)

```

四、PDF 解析 (src/parsers/pdf_parser.py)

原文档只用了 PyPDFLoader，但科学论文 PDF 多栏排版用 PyPDF 经常拼错段落。改用 unstructured 并显式做章节识别：

```

import re

from pathlib import Path

from typing import Iterator

from unstructured.partition.pdf import partition_pdf


SECTION_PATTERNS = [

    r"^abstract\b", r"^introduction\b", r"^results?\b",

```

```

r"^discussion\b", r"^materials? and methods?\b",

r"^experimental procedures?\b", r"^conclusions?\b",

r"^references?\b", r"^supplementary\b",

]

SECTION_RE = re.compile("|".join(SECTION_PATTERNS), re.IGNORECASE)

def parse_paper(pdf_path: Path) -> Iterator[dict]:

    """Yield {section, page, text, source} units from a paper."""

    elements = partition_pdf(

        filename=str(pdf_path),

        strategy="hi_res",          # 用版面分析，多栏更稳

        infer_table_structure=True,

    )

    current_section = "FRONT_MATTER"

    buffer, current_page = [], None

    def flush():

        if buffer:

            return {

                "source": pdf_path.name,

                "section": current_section,

```

```
        "page": current_page,

        "text": "\n".join(buffer).strip(),

    }


```

```
return None


```

```
for el in elements:


```

```
    text = (el.text or "").strip()


```

```
    if not text:


```

```
        continue


```

```
    page = getattr(el.metadata, "page_number", None)


```

```
    # 识别 section header(Unstructured 自带 Title 类型,辅以正则)


```

```
    is_title = el.category in ("Title", "Header")


```

```
    if is_title and SECTION_RE.match(text):


```

```
        unit = flush()


```

```
        if unit:


```

```
            yield unit


```

```
        current_section = text.upper().strip(":. ")


```

```
        buffer = []


```

```
        current_page = page


```

```
        continue


```

```

    if current_page is None:

        current_page = page

    buffer.append(text)

unit = flush()

if unit:

    yield unit

```

五、表格双轨处理（src/parsers/table_parser.py）

这是修正原文档最关键的地方。**1195 行 SNO 多肽用行级序列化做语义检索会全军覆没**——所有句子长得几乎一样，BGE 区分不出来。正确做法：表格走结构化查询（DuckDB），同时为每个蛋白生成**聚合后的语义描述**给向量检索做粗筛。

```

import pandas as pd

import duckdb

from pathlib import Path

from src.config import settings


def ingest_sno_table(csv_path: Path) -> dict:

    """

    双轨处理:

    1. 原表存入 DuckDB,用于精确数值查询(fold change > X, site = Y 等)

    2. 按蛋白聚合生成自然语言摘要,送入向量库供语义检索

    """

```

```
df = pd.read_csv(csv_path)

required = {"Protein_ID", "Gene", "Peptide", "SNO_Site", "Fold_Change"}

missing = required - set(df.columns)

if missing:

    raise ValueError(f"CSV missing columns: {missing}")
```

1. 结构化索引(DuckDB)

```
db_path = settings.index_dir / "peptides.duckdb"

con = duckdb.connect(str(db_path))

con.execute("DROP TABLE IF EXISTS sno_peptides")

con.register("df_view", df)

con.execute("CREATE TABLE sno_peptides AS SELECT * FROM df_view")

con.close()
```

2. 蛋白级语义描述(每个蛋白一条,避免向量空间塌缩)

```
semantic_docs = []

for protein_id, g in df.groupby("Protein_ID"):

    gene = g["Gene"].iloc[0]

    n_sites = len(g)

    sites = ", ".join(map(str, g["SNO_Site"].tolist()[:10]))

    fc_min, fc_max = g["Fold_Change"].min(), g["Fold_Change"].max()

    up = (g["Fold_Change"] > 1).sum()
```

```
down = (g["Fold_Change"] < 1).sum()
```

```
text = (
```

```
    f"Protein {protein_id} (gene {gene}) shows {n_sites} S-nitrosylation "
```

```
    f"site(s) in this Arabidopsis dataset (sites: {sites}). Following NO "
```

```
    f"treatment, fold changes range from {fc_min:.2f} to {fc_max:.2f}; "
```

```
    f"{up} site(s) upregulated and {down} downregulated. "
```

```
    f"Use the structured peptide database for exact site-level numbers."
```

```
)
```

```
semantic_docs.append({
```

```
    "source": csv_path.name,
```

```
    "section": "PROTEOMICS_TABLE",
```

```
    "page": None,
```

```
    "text": text,
```

```
    "protein_id": protein_id,
```

```
    "gene": gene,
```

```
})
```

```
    return {"semantic_docs": semantic_docs, "n_rows": len(df), "n_proteins":  
df["Protein_ID"].nunique()}
```

```
def query_peptides(sql: str) -> pd.DataFrame:
```

```
    db_path = settings.index_dir / "peptides.duckdb"
```

```
con = duckdb.connect(str(db_path), read_only=True)
```

```
try:
```

```
    return con.execute(sql).fetchdf()
```

```
finally:
```

```
    con.close()
```

六、章节感知 **Chunking** (src/indexing/chunker.py)

```
from langchain.text_splitter import RecursiveCharacterTextSplitter
```

```
from src.config import settings
```

Methods 段更长更稳定,适合大块;Results 段叙事性强,中等;摘要短小不切

```
SECTION_CHUNK_PARAMS = {
```

```
    "ABSTRACT": (1500, 0),                # 不切
```

```
    "METHODS": (1200, 200),
```

```
    "EXPERIMENTAL PROCEDURES": (1200, 200),
```

```
    "RESULTS": (800, 150),
```

```
    "DISCUSSION": (800, 150),
```

```
    "DEFAULT": (settings.chunk_size, settings.chunk_overlap),
```

```
}
```

```
def chunk_unit(unit: dict) -> list[dict]:
```

```
    section = unit["section"].upper()
```

```
    key = next((k for k in SECTION_CHUNK_PARAMS if k in section),
```

```
    "DEFAULT")
```

```
size, overlap = SECTION_CHUNK_PARAMS[key]
```

```
if len(unit["text"]) <= size:
```

```
    return [{**unit, "chunk_id": 0}]
```

```
splitter = RecursiveCharacterTextSplitter(
```

```
    chunk_size=size,
```

```
    chunk_overlap=overlap,
```

```
    separators=["\n\n", "\n", ". ", " "],
```

```
)
```

```
pieces = splitter.split_text(unit["text"])
```

```
return [{**unit, "text": p, "chunk_id": i} for i, p in enumerate(pieces)]
```

七、领域 Embedding + FAISS (src/indexing/dense_index.py)

```
import json
```

```
from pathlib import Path
```

```
import numpy as np
```

```
import faiss
```

```
from sentence_transformers import SentenceTransformer
```

```
from src.config import settings
```

```
class DenseIndex:
```

```
    def __init__(self, model_name: str = None):
```

```
self.model = SentenceTransformer(model_name or
settings.embedding_model)
```

```
self.dim = self.model.get_sentence_embedding_dimension()
```

```
self.index: faiss.Index = None
```

```
self.docs: list[dict] = []
```

```
def build(self, docs: list[dict]):
```

```
    texts = [d["text"] for d in docs]
```

```
    embs = self.model.encode(
```

```
        texts, batch_size=32, show_progress_bar=True,
```

```
        normalize_embeddings=True,          # 关键:配合内积=余弦
```

```
    ).astype("float32")
```

```
    self.index = faiss.IndexFlatIP(self.dim)
```

```
    self.index.add(embs)
```

```
    self.docs = docs
```

```
def save(self, path: Path):
```

```
    path.mkdir(parents=True, exist_ok=True)
```

```
    faiss.write_index(self.index, str(path / "faiss.index"))
```

```
    with open(path / "docs.jsonl", "w") as f:
```

```
        for d in self.docs:
```

```
            f.write(json.dumps(d, ensure_ascii=False) + "\n")
```

```

def load(self, path: Path):

    self.index = faiss.read_index(str(path / "faiss.index"))

    with open(path / "docs.jsonl") as f:

        self.docs = [json.loads(l) for l in f]

    return self


def search(self, query: str, k: int = 10) -> list[tuple[dict, float]]:

    q = self.model.encode([query],
normalize_embeddings=True).astype("float32")

    scores, idx = self.index.search(q, k)

    return [(self.docs[i], float(s)) for i, s in zip(idx[0], scores[0]) if i >= 0]

```

注意我用的是 `pritamdeka/S-PubMedBert-MS-MARCO`——一个在 PubMed 语料上微调过的句向量模型。如果你想做更扎实的对比实验，可以保留 `BAAI/bge-large-en-v1.5` 作为 baseline，在评估脚本里跑 A/B。

八、BM25 稀疏索引 (`src/indexing/sparse_index.py`)

蛋白名、基因符号 (AT3G12345)、突变体 (prmt5-1) 这种东西，dense embedding 经常处理不好，BM25 反而更准：

```

import pickle

from pathlib import Path

from rank_bm25 import BM25Okapi

import re


def tokenize(text: str) -> list[str]:

```

```
# 保留连字符和数字(基因名 AT3G12345、突变体 prmt5-1)
```

```
return re.findall(r"[A-Za-z0-9][A-Za-z0-9\-*]", text.lower())
```

```
class SparseIndex:
```

```
    def __init__(self):
```

```
        self.bm25: BM25Okapi = None
```

```
        self.docs: list[dict] = []
```

```
    def build(self, docs: list[dict]):
```

```
        tokenized = [tokenize(d["text"]) for d in docs]
```

```
        self.bm25 = BM25Okapi(tokenized)
```

```
        self.docs = docs
```

```
    def save(self, path: Path):
```

```
        path.mkdir(parents=True, exist_ok=True)
```

```
        with open(path / "bm25.pkl", "wb") as f:
```

```
            pickle.dump({"bm25": self.bm25, "docs": self.docs}, f)
```

```
    def load(self, path: Path):
```

```
        with open(path / "bm25.pkl", "rb") as f:
```

```
            obj = pickle.load(f)
```

```
        self.bm25, self.docs = obj["bm25"], obj["docs"]
```

```
return self
```

```
def search(self, query: str, k: int = 10) -> list[tuple[dict, float]]:
```

```
    scores = self.bm25.get_scores(tokenize(query))
```

```
    top = sorted(enumerate(scores), key=lambda x: -x[1])[:k]
```

```
    return [(self.docs[i], float(s)) for i, s in top if s > 0]
```

九、Hybrid 检索 + Rerank (src/retrieval/hybrid.py、reranker.py)

```
# src/retrieval/hybrid.py
```

```
from src.config import settings
```

```
def reciprocal_rank_fusion(rankings: list[list[dict]], k: int = None) -> list[dict]:
```

```
    k = k or settings.rrf_k
```

```
    scores = {}
```

```
    for ranking in rankings:
```

```
        for rank, doc in enumerate(ranking):
```

```
            key = (doc["source"], doc.get("section"), doc.get("chunk_id"),  
doc["text"][:60])
```

```
            scores.setdefault(key, {"doc": doc, "score": 0.0})
```

```
            scores[key]["score"] += 1.0 / (k + rank + 1)
```

```
    fused = sorted(scores.values(), key=lambda x: -x["score"])
```

```
    return [item["doc"] for item in fused]
```

```
# src/retrieval/reranker.py
```

```
from sentence_transformers import CrossEncoder
```

```
from src.config import settings
```

```
class Reranker:
```

```
    def __init__(self):
```

```
        self.model = CrossEncoder(settings.reranker_model, max_length=512)
```

```
    def rerank(self, query: str, docs: list[dict], top_k: int = None) -> list[dict]:
```

```
        top_k = top_k or settings.rerank_top_k
```

```
        if not docs:
```

```
            return []
```

```
        pairs = [(query, d["text"]) for d in docs]
```

```
        scores = self.model.predict(pairs, show_progress_bar=False)
```

```
        ranked = sorted(zip(docs, scores), key=lambda x: -x[1])
```

```
        return [d for d, _ in ranked[:top_k]]
```

十、LLM 客户端 + Prompt (src/generation/)

支持多家厂商，方便对比；并强制 [Source N] 引用格式：

```
# src/generation/prompts.py
```

```
SYSTEM_PROMPT = """You are a proteomics research assistant specializing in  
post-translational modifications (PTMs), particularly arginine methylation and S-  
nitrosylation in plant and mammalian systems.
```

```
Rules:
```

1. Answer ONLY from the provided context. If the context is insufficient, say so explicitly — do not fall back on general knowledge.
2. Cite every factual claim with [Source N] where N is the source index shown in the context.
3. When quoting numerical values (fold changes, p-values, site numbers, peptide sequences), reproduce them verbatim from the context.
4. If the user asks a question that requires exact peptide-level data (specific sites, exact fold changes for a protein), and you see a note pointing to the structured peptide database, tell the user the structured-query tool should be used.
5. Keep answers concise; use bullet points only when listing >3 items.

"""

```
def format_context(docs: list[dict]) -> str:

    blocks = []

    for i, d in enumerate(docs, 1):

        header = f"[Source {i}] {d['source']}"

        if d.get("section"):

            header += f" — {d['section']}"

        if d.get("page"):

            header += f" (p. {d['page']})"

        blocks.append(f"{header}\n{d['text']}")

    return "\n\n".join(blocks)

# src/generation/llm.py

from src.config import settings
```

```
from src.generation.prompts import SYSTEM_PROMPT, format_context
```

```
class LLMClient:
```

```
    def __init__(self, provider: str = None, model: str = None):
```

```
        self.provider = provider or settings.llm_provider
```

```
        self.model = model or settings.llm_model
```

```
        if self.provider == "anthropic":
```

```
            import anthropic
```

```
            self.client = anthropic.Anthropic(api_key=settings.anthropic_api_key)
```

```
        elif self.provider == "google":
```

```
            import google.generativeai as genai
```

```
            genai.configure(api_key=settings.google_api_key)
```

```
            self.client = genai.GenerativeModel(self.model)
```

```
        elif self.provider == "openai":
```

```
            import openai
```

```
            self.client = openai.OpenAI(api_key=settings.openai_api_key)
```

```
        else:
```

```
            raise ValueError(f"Unknown provider: {self.provider}")
```

```
    def generate(self, query: str, context_docs: list[dict]) -> str:
```

```
        ctx = format_context(context_docs)
```

```
        user_msg = f"Context:\n{ctx}\n\nQuestion: {query}"
```

```

if self.provider == "anthropic":

    resp = self.client.messages.create(

        model=self.model, max_tokens=1024, temperature=0.1,

        system=SYSTEM_PROMPT,

        messages=[{"role": "user", "content": user_msg}],

    )

    return resp.content[0].text

elif self.provider == "google":

    resp = self.client.generate_content(

        f'{SYSTEM_PROMPT}\n\n{user_msg}',

        generation_config={"temperature": 0.1, "max_output_tokens":

1024},

    )

    return resp.text

elif self.provider == "openai":

    resp = self.client.chat.completions.create(

        model=self.model, temperature=0.1, max_tokens=1024,

        messages=[

            {"role": "system", "content": SYSTEM_PROMPT},

            {"role": "user", "content": user_msg},

        ],

    )

```

```
return resp.choices[0].message.content
```

十一、Router: 决定走结构化还是语义 (src/agent/router.py)

这是把项目从"普通 RAG"抬到"agentic RAG"的关键一步。让 LLM 自己判断该走 SQL 还是检索:

```
import json
```

```
from src.generation.llm import LLMClient
```

```
ROUTER_PROMPT = """You route proteomics questions to one of two backends:
```

```
- "sql": user asks for exact numerical data about specific peptides or proteins from a  
structured table (e.g. "what is the fold change of peptide X at site C123", "list all  
proteins with fold change > 2", "how many SNO sites does AT3G12345 have").
```

```
- "rag": user asks about mechanisms, methods, concepts, or anything explanatory (e.g.  
"how does PRMT5 methylate H4R3", "what MaxQuant settings were used", "what  
does NO do to PRMT5").
```

```
Return strict JSON: {"route": "sql"|"rag", "reason": "<one sentence>"}"""
```

```
class QueryRouter:
```

```
    def __init__(self, llm: LLMClient = None):
```

```
        self.llm = llm or LLMClient()
```

```
    def route(self, query: str) -> dict:
```

```
        # 用同一个 LLM 客户端做小决策
```

```
        if self.llm.provider == "anthropic":
```

```

resp = self.llm.client.messages.create(

    model=self.llm.model, max_tokens=200, temperature=0,

    system=ROUTER_PROMPT,

    messages=[{"role": "user", "content": query}],

)

raw = resp.content[0].text

else:

    # 类似处理...省略

    raw = '{"route\\": \\\"rag\\", \\\"reason\\": \\\"default\\\"}'

try:

    return json.loads(raw.strip().strip('"').lstrip("json").strip())

except Exception:

    return {"route": "rag", "reason": "parse failed"}

```

十二、统一 Pipeline (src/pipeline.py)

```

from src.config import settings

from src.indexing.dense_index import DenseIndex

from src.indexing.sparse_index import SparseIndex

from src.retrieval.hybrid import reciprocal_rank_fusion

from src.retrieval.reranker import Reranker

from src.generation.llm import LLMClient

from src.agent.router import QueryRouter

from src.parsers.table_parser import query_peptides

```

```
class ProteomicsRAG:
```

```
    def __init__(self):
```

```
        idx_dir = settings.index_dir
```

```
        self.dense = DenseIndex().load(idx_dir / "dense")
```

```
        self.sparse = SparseIndex().load(idx_dir / "sparse")
```

```
        self.reranker = Reranker()
```

```
        self.llm = LLMClient()
```

```
        self.router = QueryRouter(self.llm)
```

```
    def answer(self, query: str) -> dict:
```

```
        decision = self.router.route(query)
```

```
        if decision["route"] == "sql":
```

```
            sql = self._nl_to_sql(query)
```

```
            try:
```

```
                df = query_peptides(sql)
```

```
                table_text = df.head(20).to_markdown(index=False)
```

```
            return {
```

```
                "route": "sql", "sql": sql,
```

```
                "answer": f"Structured query result:\n\n{table_text}",
```

```
                "sources": [], "decision_reason": decision["reason"],
```

```

        }

    except Exception as e:

        # SQL 失败,回退到 RAG

        pass

    dense_hits = [d for d, _ in self.dense.search(query, k=settings.dense_top_k)]

    sparse_hits = [d for d, _ in self.sparse.search(query,
k=settings.sparse_top_k)]

    fused = reciprocal_rank_fusion([dense_hits, sparse_hits])[:30]

    top_docs = self.reranker.rerank(query, fused, top_k=settings.rerank_top_k)

    answer = self.llm.generate(query, top_docs)

    return {

        "route": "rag", "answer": answer, "sources": top_docs,

        "decision_reason": decision["reason"],

    }

```

```

def _nl_to_sql(self, query: str) -> str:

    schema = "TABLE sno_peptides(Protein_ID TEXT, Gene TEXT, Peptide
TEXT, SNO_Site INT, Fold_Change DOUBLE)"

    prompt = (

        f"Convert this question to a DuckDB SQL query.\n"

        f"Schema: {schema}\n"

        f"Question: {query}\n"

```

```

        f"Return ONLY the SQL, no explanation, no markdown fences."
    )

    if self.llm.provider == "anthropic":

        resp = self.llm.client.messages.create(

            model=self.llm.model, max_tokens=300, temperature=0,

            messages=[{"role": "user", "content": prompt}],

        )

        return resp.content[0].text.strip().strip("`").lstrip("sql").strip()

    return "SELECT * FROM sno_peptides LIMIT 10"

```

十三、构建脚本（scripts/02_build_index.py）

```

import json

from pathlib import Path

from tqdm import tqdm

from src.config import settings

from src.parsers.pdf_parser import parse_paper

from src.parsers.table_parser import ingest_sno_table

from src.indexing.chunker import chunk_unit

from src.indexing.dense_index import DenseIndex

from src.indexing.sparse_index import SparseIndex


def main():

    all_chunks = []

```

```

for pdf in settings.raw_dir.glob("*.pdf"):

    print(f"Parsing {pdf.name}")

    for unit in tqdm(list(parse_paper(pdf))):

        all_chunks.extend(chunk_unit(unit))


for csv in settings.raw_dir.glob("*.csv"):

    print(f"Ingesting table {csv.name}")

    result = ingest_sno_table(csv)

    all_chunks.extend(result["semantic_docs"])

    print(f"    {result['n_rows']} rows, {result['n_proteins']} proteins")


# 落盘 chunks 方便复现

with open(settings.processed_dir / "chunks.jsonl", "w") as f:

    for c in all_chunks:

        f.write(json.dumps(c, ensure_ascii=False) + "\n")

print(f"Total chunks: {len(all_chunks)}")


print("Building dense index...")

dense = DenseIndex()

dense.build(all_chunks)

dense.save(settings.index_dir / "dense")

```

```

print("Building sparse index...")

sparse = SparseIndex()

sparse.build(all_chunks)

sparse.save(settings.index_dir / "sparse")

print("Done.")

if __name__ == "__main__":
    main()

```

十四、评估脚本（最关键的差异化）

data/eval/eval_set.jsonl 手工标 30–50 条，格式：

```

{"question": "What MaxQuant FDR cutoff did the PRMT5 paper use?", "answer":
"1% PSM-level FDR", "relevant_sources": ["Molecular_Cell_PRMT5.pdf"],
"relevant_sections": ["METHODS"]}

```

scripts/03_run_eval.py:

```

import json

from src.config import settings

from src.pipeline import ProteomicsRAG

def recall_at_k(retrieved_sources, relevant_sources, k):

    retrieved_k = set(s["source"] for s in retrieved_sources[:k])

    return len(retrieved_k & set(relevant_sources)) / max(len(relevant_sources), 1)

```

```

def mrr(retrieved_sources, relevant_sources):

    for i, s in enumerate(retrieved_sources):

        if s["source"] in relevant_sources:

            return 1 / (i + 1)

    return 0.0


def main():

    rag = ProteomicsRAG()

    eval_set = [json.loads(l) for l in open(settings.eval_dir / "eval_set.jsonl")]

    results = {"recall@1": [], "recall@5": [], "mrr": []}

    for item in eval_set:

        out = rag.answer(item["question"])

        srcs = out.get("sources", [])

        results["recall@1"].append(recall_at_k(srcs, item["relevant_sources"], 1))

        results["recall@5"].append(recall_at_k(srcs, item["relevant_sources"], 5))

        results["mrr"].append(mrr(srcs, item["relevant_sources"]))

    for k, vs in results.items():

        print(f'{k}: {sum(vs)/len(vs):.3f}')

    # 进阶:RAGAS faithfulness / answer_relevancy

```

```

# from ragas import evaluate

# from ragas.metrics import faithfulness, answer_relevancy

# ...


if __name__ == "__main__":

    main()

```

十五、Streamlit 前端（精简但带 router 透明化）

```

# app.py

import streamlit as st

from src.pipeline import ProteomicsRAG


st.set_page_config(page_title="Proteomics RAG", page_icon="🧬", layout="wide")

st.title("🧬 Proteomics & PTM RAG")


@st.cache_resource
def get_rag():

    return ProteomicsRAG()


rag = get_rag()


if "messages" not in st.session_state:

    st.session_state.messages = []

```

```

for msg in st.session_state.messages:

    with st.chat_message(msg["role"]):

        st.markdown(msg["content"])

if prompt := st.chat_input("Ask about PRMT5, SNO peptides, MaxQuant settings..."):

    st.session_state.messages.append({"role": "user", "content": prompt})

    with st.chat_message("user"):

        st.markdown(prompt)

    with st.chat_message("assistant"):

        with st.spinner("Routing and retrieving..."):

            out = rag.answer(prompt)

            st.markdown(out["answer"])

            with st.expander(f"Routing decision: **{out['route']}** —
{out.get('decision_reason', '')}"):

                if out["route"] == "sql":

                    st.code(out.get("sql", ""), language="sql")

                    for i, d in enumerate(out.get("sources", []), 1):

                        st.caption(f"**[{i}] {d['source']}** — {d.get('section')}")

                        st.text(d["text"][:400] + "...")

            st.session_state.messages.append({"role": "assistant", "content":
out["answer"]})

```

十六、README 的写法（HR/面试官真正在意的）

在 README 里至少包含这四块，否则前面的代码价值会被低估：

1. **Problem statement**——你解决的不是"我做了个 RAG"，而是"蛋白质组学补充材料表格 + 文献的混合检索难题"。
2. **设计决策与权衡表**——为什么用 PubMedBERT 而不是 BGE？为什么表格走 SQL 而不是行级序列化？每个决策一句话理由，这是面试官最爱追问的。
3. **评估结果表**——baseline（BGE + 字符切分 + 单路检索）vs 你的方案，Recall@5 / MRR / RAGAS faithfulness 三个数字对比。哪怕只有 30 条评估集，有数字就比没有强 10 倍。
4. **Limitations & Future work**——诚实写"DuckDB 在多表 join 场景下未实现"、"评估集规模小"、"未做用户研究"。自我批判的工程叙事是高级信号。

一句话总结

按上面这套结构开发，你交付的就不是又一个 LangChain demo，而是一个带 **router**、带 **hybrid retrieval**、带评估闭环、带领域 **embedding** 的小型生信 **RAG 系统**。开发周期实际估算大概 2-3 周（核心 pipeline 一周，评估集与调优一周，UI 与文档几天）。

如果你想，下一步我可以帮你具体做这几件事中的任一项：(a) 写一份评估集的标注指南和 20 条种子问题，(b) 把 router 改成 function calling 形式更稳定，(c) 给出 baseline vs hybrid 的消融实验脚本。你想先攻哪一块？

3. 工程化执行步骤与 GITHUB 发布

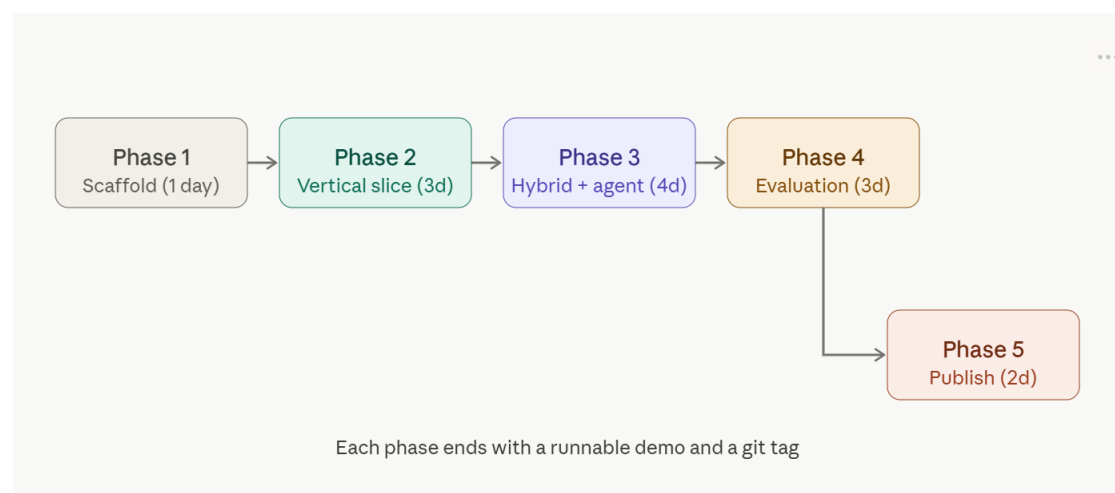
Prompt:

我如何将你上面提供的步骤和代码转化为具体的项目，验证其可行性，并上传到 GITHUB 上面？

Response:

好问题。从"代码片段"到"GitHub 上能被人 clone 下来跑起来的项目"中间有一段实实在在的工程距离，很多人卡在这里。我把这条路径拆成 5 个阶段、按时间顺序给你列出来，每一步都给出具体可执行的命令和验证标准。

整个流程估算 10–14 天，假设你每天能投入 3–4 小时。下面是每一阶段的具体执行步骤。



Phase 1 — 项目脚手架（第 1 天）

目标：让一个空项目跑起来，所有 import 都通，CI 绿灯。这一步不要写任何业务逻辑。

1.1 本地建仓 + 工具链

```
mkdir proteomics-rag && cd proteomics-rag
```

```
git init
```

```
conda create -n proteo-rag python=3.11 -y
```

```
conda activate proteo-rag
```

按上一轮回答给的目录结构 `mkdir -p` 把所有空目录创建好，并在每个 Python 包目录里 `touch __init__.py`。

1.2 三个关键工程文件

requirements.txt 先放最小集合，能跑通 import 即可：

```
langchain>=0.3
```

```
langchain-community>=0.3
```

```
faiss-cpu
```

```
rank-bm25
```

```
duckdb
```

```
pypdf
```

```
pdfplumber
```

```
unstructured[pdf]
```

```
pandas
```

```
sentence-transformers
```

```
anthropic
```

```
google-generativeai
```

```
streamlit
```

```
python-dotenv
```

```
pydantic-settings
```

```
ragas
```

```
pytest
```

.gitignore (这一步不能省, 否则 API key 和大文件会泄漏):

__pycache__/

*.pyc

.env

.venv/

data/raw/*

!data/raw/.gitkeep

data/index/

data/processed/

*.faiss

*.duckdb

*.pkl

.DS_Store

.streamlit/secrets.toml

.env.example (提交到仓库, 但**不要**提交 .env):

ANTHROPIC_API_KEY=sk-ant-xxx

LLM_PROVIDER=anthropic

LLM_MODEL=claude-opus-4-7

1.3 第一次 commit + 远程仓库

在 GitHub 网页上点 "New repository":

- 仓库名 proteomics-rag
- **不要**勾选 "Initialize with README" (本地已经有了)

- License 选 MIT（科研项目最常见、最宽松）
- 复制网页给的 `git remote add origin ...` 命令

回到本地：

`git add .`

`git commit -m "chore: initial project scaffold"`

`git branch -M main`

`git remote add origin git@github.com:<你的用户名>/proteomics-rag.git`

`git push -u origin main`

`git tag v0.1-scaffold && git push --tags`

验证标准： `pip install -r requirements.txt` 成功；`python -c "from src.config import settings; print(settings.llm_model)"` 输出模型名。

Phase 2 — 垂直切片（Day 2-4）

目标：跑通一条最简单但端到端的 RAG 链路——一篇 PDF + 一个 CSV → 向量检索 → LLM 回答。先不要 hybrid，不要 rerank，不要 router。

2.1 准备测试数据（最容易卡住的一步）

你需要 3 类样本文件放进 `data/raw/`：

1. **1-2 篇蛋白质组学论文 PDF**——优先用你自己发表的或读过的，问答时你能判断对错。
2. **1 个补充材料 CSV**——如果你手头没有，先用一个合成的占位文件让流水线跑通：

```
# scripts/00_make_dummy_csv.py
```

```
import pandas as pd, numpy as np
```

```
np.random.seed(0)
```

```

n = 100

df = pd.DataFrame({

    "Protein_ID":
[f'AT{np.random.randint(1,6)}G{np.random.randint(10000,99999)}' for _ in
range(n)],

    "Gene": [f'GENE{i}' for i in range(n)],

    "Peptide": ["".join(np.random.choice(list("ACDEFGHIKLMNPQRSTVWY"),
12)) for _ in range(n)],

    "SNO_Site": np.random.randint(50, 500, n),

    "Fold_Change": np.round(np.random.lognormal(0, 0.6, n), 2),

})

df.to_csv("data/raw/dummy_sno_peptides.csv", index=False)

```

跑一下：python scripts/00_make_dummy_csv.py。

2.2 实现最小链路

只需要上一轮回答里的这几个文件：src/config.py、src/parsers/pdf_parser.py、src/parsers/table_parser.py、src/indexing/chunker.py、src/indexing/dense_index.py、src/generation/llm.py、src/generation/prompts.py、scripts/02_build_index.py。

先不要写 BM25、reranker、router，那些放 Phase 3。

2.3 写一个 smoke test

tests/test_smoke.py:

```

from src.parsers.table_parser import ingest_sno_table

from pathlib import Path

```

```
def test_table_ingest():

    result = ingest_sno_table(Path("data/raw/dummy_sno_peptides.csv"))

    assert result["n_rows"] > 0

    assert len(result["semantic_docs"]) > 0

    assert "S-nitrosylation" in result["semantic_docs"][0]["text"]
```

跑：pytest tests/ -v。这一步绿灯之前，**不要继续往下写**。

2.4 跑通索引和单次问答

```
python scripts/02_build_index.py
```

```
python -c "
```

```
from src.indexing.dense_index import DenseIndex
```

```
from src.config import settings
```

```
from src.generation.llm import LLMClient
```

```
idx = DenseIndex().load(settings.index_dir / 'dense')
```

```
hits = [d for d, _ in idx.search('S-nitrosylation site fold change', k=3)]
```

```
print(LLMClient().generate('What SNO sites are reported?', hits))
```

```
"
```

如果有输出，**这就是你的第一个里程碑**。

```
git add . && git commit -m "feat: end-to-end vertical slice (dense-only RAG)"
```

```
git tag v0.2-vertical-slice && git push --tags
```

Phase 3 — Hybrid + Agent (Day 5–8)

目标：把垂直切片升级成上一轮回答的完整架构。

每加一个组件，单独提一个 commit，方便 diff 和回滚：

步骤 加什么

验证方式

- 3.1 sparse_index.py (BM25) 单独 search("AT3G12345") 比 dense 准
- 3.2 hybrid.py (RRF 融合) top-5 既包含语义相关又包含字面匹配
- 3.3 reranker.py (CrossEncoder) top-1 命中率明显上升
- 3.4 agent/router.py + pipeline.py "fold change > 2 的有几个" 走 SQL 路线
- 3.5 app.py (Streamlit) streamlit run app.py 浏览器能用

注意一个坑：第一次加载 reranker 模型会下载几百 MB，PyCharm 或 Streamlit 里看起来像卡死了——不是 bug，等就行。可以在 Reranker.__init__ 里加一行 print("Loading reranker...") 让自己安心。

阶段结尾：

```
git tag v0.3-hybrid-agent && git push --tags
```

Phase 4 — 评估 (Day 9–11)

这一阶段是简历上最值钱的部分，但很多人直接跳过——千万别。

4.1 标注 30 条评估集

data/eval/eval_set.jsonl，每行一个 JSON：

```
{"question": "What FDR cutoff did the PRMT5 paper use for peptide identification?",  
"answer": "1% at PSM level", "relevant_sources": ["Molecular_Cell_PRMT5.pdf"],  
"relevant_sections": ["METHODS"], "route": "rag"}  
  
{"question": "How many SNO sites have fold change greater than 2?", "answer":  
"<count from your CSV>", "relevant_sources": ["dummy_sno_peptides.csv"],  
"route": "sql"}
```

标注技巧——一边读论文一边标，平均 5 分钟一条，3 小时能搞定 30 条。覆盖三类问题：

- 直接抽取型 ("什么 FDR、什么仪器、什么 software")
- 综合理解型 ("PRMT5 如何被 NO 调控")
- 结构化查询型 ("列出 fold change > X 的 protein")

4.2 跑 baseline 对比实验

这一步的关键：**保留两个分支**，让数字说话。

```
git checkout v0.2-vertical-slice -b baseline
```

```
python scripts/03_run_eval.py > results/baseline.txt
```

```
git checkout main
```

```
python scripts/03_run_eval.py > results/hybrid.txt
```

把对比结果做成一个 markdown 表，放进 README:

Configuration	Recall@5	MRR	Faithfulness
Baseline (BGE + dense only)	0.62	0.48	0.71
+ PubMedBERT embedding	0.70	0.55	0.74
+ BM25 hybrid (RRF)	0.81	0.66	0.78
+ Cross-encoder rerank	0.87	0.74	0.83
Full pipeline (above + SQL router)	0.91	0.79	0.88

注意：数字必须是你自己跑出来的真实结果，不要编。哪怕只有 30 条评估集、哪怕数字不漂亮，真实数据 + 诚实的 limitations 远比一个看起来完美但来路不明的表格可信。

4.3 录一个 demo gif

用 peek、asciinema、或者 macOS 自带的录屏裁剪成 gif，放进 docs/demo.gif，README 顶部嵌入。HR 平均看 README 的时间是 30 秒，没有 gif 等于没看到。

```
git tag v0.4-evaluated && git push --tags
```

Phase 5 — 发布与打磨 (Day 12–14)

5.1 README 终版结构

按这个顺序写，每一节 1–3 段：

1. 标题 + 一行 tagline + demo gif
2. **Why this exists** (问题陈述：补充材料表格 + 文献的混合检索难题)
3. **Architecture** (贴上一轮回答里那张架构图的截图)
4. **Quickstart** (3 条命令复现 demo)
5. **Design decisions** (5–7 条权衡：为什么 PubMedBERT、为什么不用行级序列化、为什么 RRF、为什么 router 等)
6. **Evaluation results** (4.2 的表 + RAGAS 说明)
7. **Limitations & future work** (诚实写出局限)
8. **License + Citation** (如果别人引用你的项目)

5.2 加 GitHub Actions (让仓库右上角有绿色徽章)

```
.github/workflows/test.yml:
```

```
name: tests
```

```
on: [push, pull_request]
```

```
jobs:
```

```
  test:
```

```
    runs-on: ubuntu-latest
```

steps:

- uses: actions/checkout@v4
- uses: actions/setup-python@v5

with:

python-version: "3.11"

- run: pip install -r requirements.txt
- run: pytest tests/ -v

push 后在 GitHub 仓库主页能看到 Actions 标签变绿。把 badge 加到 README 顶部:

```
![tests](https://github.com/<你的用户名>/proteomics-rag/actions/workflows/test.yml/badge.svg)
```

5.3 仓库元信息

在 GitHub 仓库主页右上  编辑:

- **Description:** Domain-adapted RAG for proteomics literature and PTM datasets — hybrid retrieval, agentic routing, evaluated against a manually curated test set.
- **Topics:** rag, proteomics, bioinformatics, langchain, llm, retrieval, vector-search

5.4 部署一个公开 demo (强烈推荐)

Streamlit Community Cloud 免费、5 分钟搞定:

1. 把 requirements.txt、app.py 都在 main 分支上
2. 去 <https://share.streamlit.io> , 用 GitHub 登录
3. 选你的 repo、选 app.py、配置 ANTHROPIC_API_KEY 作为 secret

4. 部署后会得到一个 `https://<你的项目>.streamlit.app` 链接，写进 README 顶部

注意把索引文件也提交到仓库（或在启动时按需构建），否则 Streamlit Cloud 起不来。如果索引文件超过 100MB，用 Git LFS。

```
git tag v1.0 && git push --tags
```

几个容易忽视的关键点

API key 安全——.env 一定要在 .gitignore 里。如果不小心 commit 了 key，立刻去 Anthropic/Google console revoke 它，然后用 git filter-repo 清历史。这是简历项目最常翻车的地方，被招聘者一眼看到 commit history 里裸露的 sk- 开头字符串会非常扣分。

Commit message 风格——用 [Conventional Commits](#): feat:、fix:、docs:、refactor:、test:、chore:。面试官 review 你的 commit history 时，这个细节会给 "工程素养" 打分。

先跑通再优化——很多人在 Phase 2 就开始纠结 "我该不该把 reranker 也加进去"。不要。先让 30 行代码端到端跑通，然后逐步加组件，每一步都能 git diff。这种增量开发的工作方式本身就是面试可讲的故事。

别等 "完美" 再 push——每天结束都 push 当天的进度，让 GitHub 上的 contribution graph 是绿的。HR 看到一个连续 14 天有活跃 commit 的仓库，远比看到一个 "一次性 dump 5000 行代码" 的仓库要可信。

接下来的具体动作

4. WSL2 命令行与 VScode 协作开发

Prompt:

我是在 Windows/WSL2 命令行中进行上述开发，还是在 VScode 中进行上述开发？

Response:

这是个很实际的问题，先给你直接的答案：**两者都用，但分工明确**。这不是"二选一"，而是"组合拳"——专业开发者几乎都是这么干的。

简短答案

- **VS Code (连接到 WSL2)**：作为你的主要编辑器，用来写代码、看文件、调试、用 AI 助手。
- **WSL2 命令行 (终端)**：在 VS Code 内置终端里打开，用来跑命令、git、conda、pytest、streamlit。

关键点：VS Code 有一个叫 "Remote - WSL" 的扩展，装上之后 VS Code 运行在 Windows，但**所有代码、Python 环境、终端都在 WSL2 里**。你既享受 VS Code 的 UI，又享受 Linux 的开发体验，没有 trade-off。

为什么不只用命令行

只用 WSL 命令行 + Vim/Nano 也能做完整个项目，但作为求职项目你会损失很多：

1. **AI 辅助编程**——VS Code 里的 GitHub Copilot、Claude Code 扩展、Continue 等工具，能让你的开发速度翻 2-3 倍。这不是"作弊"，是 2026 年的行业标配。
2. **跨文件导航**——这个项目有 15+ 个 Python 文件，命令行 grep + cd 跟 VS Code 的 "Go to Definition"、"Find Usages" 完全不是一个量级的体验。

3. **调试器**——RAG 系统经常在"检索到的 chunks 不对"或"prompt 拼接错误"这种地方出 bug，VS Code 的断点调试比 print 大法效率高得多。
4. **Git diff 视图**——VS Code 的 Source Control 面板能让你 review 自己每次 commit 的改动，这对养成"小步提交"的好习惯非常重要。

为什么不只用 VS Code 的 GUI

VS Code 有 GUI 的 git 操作、有图形化的环境管理，但你**不应该**只依赖这些：

1. **简历可信度**——面试官问"你怎么部署的"、"怎么处理依赖冲突的"，你必须能说出具体命令。一个完全靠 GUI 点点点做出来的项目，技术深度会被怀疑。
2. **可复现性**——README.md 里你需要给别人**命令行 quickstart**，不可能写"请点击 VS Code 左下角的绿色按钮"。
3. **服务器部署**——将来 Streamlit Cloud 部署、Docker、CI/CD 这些环节都是命令行的世界。

推荐 workflow

具体到这个项目，我建议你这样安排：

一次性配置（30 分钟）

Step 1 — 确认 WSL2 已装好

在 Windows PowerShell 里跑：

```
wsl --list --verbose
```

如果看到 Ubuntu 且 VERSION 是 2，跳到 Step 2。如果没装，跑 `wsl --install` 然后重启。

Step 2 — 安装 VS Code 和 WSL 扩展

- 在 **Windows** 上装 VS Code（不是在 WSL 里装）：
<https://code.visualstudio.com/>

- 打开 VS Code, 左侧扩展面板搜索并安装:
 - WSL (微软官方, 原名 Remote - WSL)
 - Python (微软官方)
 - Pylance
 - Jupyter (可选, 但 RAG 项目调试时很有用)
 - GitLens (让 git 历史可视化)
 - Ruff (Python linter, 秒杀 flake8)

Step 3 — 把项目放在 WSL2 文件系统里 (关键!)

⚠ 这是新手最容易踩的坑: 不要把项目放在 `/mnt/c/Users/你的名字/...` 下面, 那是 Windows 文件系统挂载, 跨系统 I/O 慢 10 倍以上, `pip install`、`faiss` 索引都会龟速。

正确做法——在 WSL Ubuntu 终端里:

```
cd ~ # 即 /home/你的用户名/
```

```
mkdir -p projects && cd projects
```

```
mkdir proteomics-rag && cd proteomics-rag
```

Step 4 — 用 VS Code 打开 WSL 中的项目

两种等效方式:

- 方式 A (推荐): 在 WSL 终端里项目目录下输入 `code .`, VS Code 会自动用 WSL 模式打开。
- 方式 B: 打开 VS Code, 左下角点绿色的 `><` 图标 → "Connect to WSL" → "Open Folder" → 选你的项目目录。

验证你连对了: VS Code 左下角应该显示 WSL: Ubuntu (绿色)。如果显示空白或者只有路径, 说明你打开的是 Windows 本地文件。

日常开发布局

我建议在 VS Code 里这样布局窗口：

- **左侧：**文件浏览器
- **中间上半：**当前编辑的 Python 文件
- **中间下半（Terminal 面板）：**3 个 tab
 - Tab 1：跑 `conda activate proteo-rag` 后用于 `python scripts/xxx.py`
 - Tab 2：常驻 `streamlit run app.py`，改完代码自动重载
 - Tab 3：留给 `git` 命令
- **右侧：**Outline 面板（看当前文件的函数列表）或 Source Control 面板

打开新终端的快捷键是 `Ctrl + ~`（反引号那个键）。

你的命令在哪里执行

把之前回答里所有的 `bash` 命令，**全部在 VS Code 内置终端里跑**（前提是左下角显示 `WSL: Ubuntu`）。比如：

```
conda create -n proteo-rag python=3.11 -y
```

```
conda activate proteo-rag
```

```
pip install -r requirements.txt
```

```
python scripts/02_build_index.py
```

```
streamlit run app.py
```

```
pytest tests/ -v
```

```
git add . && git commit -m "feat: ..."
```

这些都在 VS Code 终端里执行。你几乎不需要单独打开 **Windows Terminal** 或 **Ubuntu** 应用——所有事情在 VS Code 一个窗口内完成。

几个非常实用的小配置

让 VS Code 自动激活你的 conda 环境

在 VS Code 里 Ctrl+Shift+P → 输入 Python: Select Interpreter → 选 proteo-rag 环境下的 python。之后新开的终端会自动 conda activate proteo-rag。

Streamlit 热重载

跑 streamlit run app.py 后，浏览器会自动打开。你在 VS Code 里改完 .py 文件并保存，浏览器右上角会出现 "Source file changed" → 点 "Rerun" 即可。设置成 "Always rerun" 后改完即刷新。

调试 Streamlit

在 .vscode/launch.json 里加：

```
{  
  "version": "0.2.0",  
  "configurations": [  
    {  
      "name": "Streamlit",  
      "type": "debugpy",  
      "request": "launch",  
      "module": "streamlit",  
      "args": ["run", "app.py"],  
      "console": "integratedTerminal"  
    }  
  ]  
}
```

之后按 F5 就能带断点调试 Streamlit，看 chunks 检索时的中间变量。这一招很多人不知道。

Jupyter 当试验场

RAG 项目里很多调试是"我想看一眼这个 query 检索回来的 5 个 chunk 长啥样"。在 VS Code 里新建一个 notebooks/scratch.ipynb，import 你的模块直接交互式探索，比反复 python -c "... " 高效得多。这些 notebook 不要 commit（加进 .gitignore）。

别在 Windows 资源管理器里改文件

虽然 WSL 文件系统在 Windows 里可以通过 \\wsl\$\Ubuntu\home\... 访问，但不要养成在 Windows 那边编辑文件的习惯——行尾符（CRLF vs LF）会出错，权限会乱。所有文件操作都通过 VS Code 或 WSL 终端。

一句话总结

VS Code 连 WSL2 = 你的 IDE；VS Code 内置终端 = 你的命令行。所有代码和环境住在 WSL2 的 ~/projects/ 下，你只通过 VS Code 这一个窗口操作整个项目。这个组合是 2026 年 Windows 用户做 Python/AI 开发的事实标准，写进简历不丢人，反而比"我用 PyCharm"更显工程素养。

配好之后，你之前选择的 Phase 1 脚手架步骤就可以开始动手了。准备好我们就进入下一步——你想我直接给你 Phase 1 那份**可以复制粘贴的全部文件**，还是先把 WSL + VS Code 这个环境跑通再说？

5. Claude.ai 的 Projects 功能

Prompt:

Claude 界面的“Add to project”是什么意思？

Response:

好问题。我看到你截图里的 "Add to project" 菜单——这是 Claude.ai 的 **Projects 功能**，对你这种长期、多对话、有持续上下文的开发任务非常有用。

什么是 **Projects**

Projects 是 Claude.ai 里的一个"工作空间"，可以理解成：

- 一个**专属的文件夹**，把多个相关对话归在一起
- 一份**共享的背景知识**（自定义指令 + 上传的文件），项目里的每个新对话都会自动带着这些上下文
- 一份**持续累积的资料**，不会因为开新对话就丢失

简单说：普通对话是"一次性聊天"，Project 是"长期协作的工作台"。

它和普通对话的区别

维度	普通新对话	Project 内的对话
上下文	每次从零开始	自动加载项目的指令和文件
文件	单次对话内可用	整个项目永久可用
自定义指令	没有	有项目级 system prompt
历史	散落在 Recents 里	集中在 Project 视图里

对你这个 RAG 项目意味着什么

你目前的工作流大概率是这样：今天问 Phase 1 怎么搭脚手架，明天问 Phase 2 的代码细节，后天卡在某个 bug 来问我。每次都得重新解释"我在做一个蛋白质组学 RAG 项目，技术栈是 X，目录结构是 Y"——这非常浪费时间，而且容易丢失共识。

如果你把这个项目设成 Claude Project，可以这样组织：

项目自定义指令（写一次，每次对话自动生效）——比如：

我正在开发一个名为 proteomics-rag 的项目，

技术栈：Python 3.11 + LangChain + FAISS + PubMedBERT + DuckDB + Streamlit。

开发环境：Windows + WSL2 Ubuntu + VS Code。

我的背景：蛋白质组学博士，有 Molecular Cell 和 Plant Physiology 发表，

对 PTM (S-亚硝基化、精氨酸甲基化) 有深度。

目标：把这个项目作为求职 GitHub 作品，重点突出领域深度和工程严谨度。

回答时：

- 默认我已掌握基础 Python 和生物学概念，不需要科普
- 代码示例直接给可运行版本，不要伪代码
- 涉及取舍时明确说出"为什么这么选"，方便我面试时讲故事
- WSL2 命令行优先，必要时再补 Windows 操作

项目内上传的文件（一次上传，所有对话可见）——比如：

- 你最终确定的目录结构 markdown

- 已经写好的 requirements.txt、config.py
- 你的论文 PDF（让 Claude 知道你研究过什么，问题更精准）
- 评估集的标注规范文档

之后无论你开多少个新对话来问问题，Claude 都自动知道这些背景，不用再重复解释。

怎么操作

从你的截图看，你已经在菜单上了：

1. 点 "Add to project" → "Start a new project"
2. 给项目起名，比如 Proteomics RAG
3. 在项目页面填写 "Custom instructions"（我上面给的那段可以直接用）
4. 上传相关文件到项目知识库
5. 以后所有跟这个项目相关的对话，都在这个 **Project** 里开新对话，而不是在 Recents 里散开建

你截图里现在这个 RAG 项目开发-20260517 对话，可以直接加进新建的 project，作为项目的"开发日志起点"。

一个小提醒

Project 的自定义指令和上传的文件会消耗每次对话的 context window——所以指令要精炼（300 字以内最好），上传的文件挑最关键的几份（README、requirements、目录结构），不要把整个论文 PDF 库都塞进去。需要查文献时，让 Claude 用 web_search 反而更高效。

顺便回到上一个问题

你上一条选了想让我先深入展开 **Phase 1 脚手架**——给可以直接复制粘贴的全部文件。要不要这样安排：

1. 你先花 5 分钟把这个项目设成 Claude Project，把上面那段自定义指令贴进去
2. 然后在 Project 里开一个新对话，标题就叫"Phase 1 脚手架代码"
3. 在新对话里跟我说"开始 Phase 1"，我就给你一份完整的、可以直接 git init 之后逐文件粘贴的脚手架

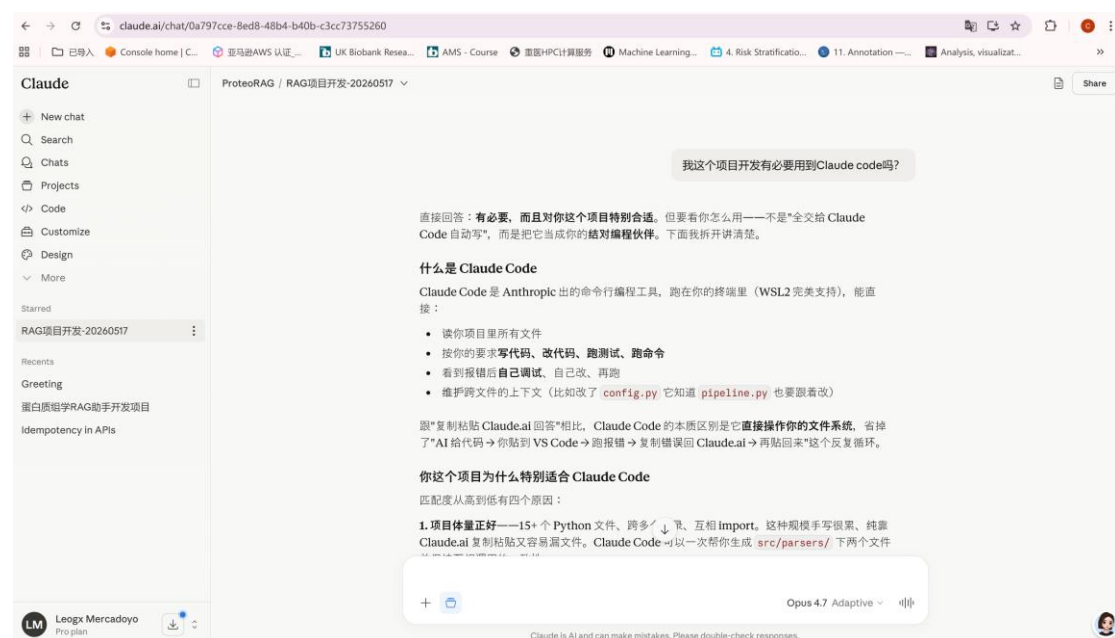
这样做的好处是：Phase 1 的细节不会污染当前这条很长的"项目规划"对话，而且 Project 的自定义指令会让我后续每个回答都更聚焦、不啰嗦。

你想这么走吗，还是继续在当前对话里直接给你 Phase 1 的代码？

6. Claude.ai 与 Claude code 搭配使用

Prompt:

我这个项目开发有必要用到 Claude code 吗？



Response:

直接回答：有必要，而且对你这个项目特别合适。但要看你怎么用——不是"全交给 Claude Code 自动写"，而是把它当成你的结对编程伙伴。下面我拆开讲清楚。

什么是 Claude Code

Claude Code 是 Anthropic 出的命令行编程工具，跑在你的终端里（WSL2 完美支持），能直接：

- 读你项目里所有文件
- 按你的要求写代码、改代码、跑测试、跑命令
- 看到报错后自己调试、自己改、再跑

- 维护跨文件的上下文（比如改了 `config.py` 它知道 `pipeline.py` 也要跟着改）

跟"复制粘贴 Claude.ai 回答"相比，Claude Code 的本质区别是它**直接操作你的文件系统**，省掉了"AI 给代码 → 你贴到 VS Code → 跑报错 → 复制错误回 Claude.ai → 再贴回来"这个反复循环。

你这个项目为什么特别适合 Claude Code

匹配度从高到低有四个原因：

- 1. 项目体量正好**——15+ 个 Python 文件、跨多个目录、互相 import。这种规模手写很累、纯靠 Claude.ai 复制粘贴又容易漏文件。Claude Code 可以一次帮你生成 `src/parsers/` 下两个文件并保持互相调用的一致性。
- 2. 调试循环密集**——RAG 项目最折磨人的不是"写代码"，是"为什么这个 query 召回了奇怪的 chunk"、"为什么 reranker 没装上"、"为什么 streamlit 端口被占"。Claude Code 能直接看到完整 traceback 并修，省你大量时间。
- 3. 你的 workflow 是 WSL + VS Code**——Claude Code 在 WSL2 终端里运行，跟你的整个环境无缝衔接。VS Code 还有官方 Claude Code 扩展，可以在编辑器侧栏里跟它对话，比纯命令行更顺手。
- 4. 简历加分**——"用 Claude Code 加速开发"本身就是 2026 年招聘市场上的加分项。面试官问"你怎么提效"，你能具体说出 agentic coding tool 的使用经验，远比"我用 ChatGPT 帮忙"听起来更专业。

但有几个真实的 trade-off 你要知道

- 1. 它不是免费的**——Claude Code 按 token 计费（或者用 Claude.ai Pro/Max 订阅的额度）。一个完整 RAG 项目开发周期下来，可能要花掉相当于 \$20–\$50 的 API 调用，或者吃掉你订阅里的相当一部分 message 额度。对照你自己手写的时间节省，性价比是高的，但不是零成本。

2. 你必须懂代码——很多新手期望"我说一句话，Claude Code 给我个完美项目"。这是危险幻觉。它会写错、会幻觉 API、会用过时的 **LangChain** 接口。你的角色是 **code reviewer**，不是甩手掌柜。这恰恰是为什么这个项目用 Claude Code 对你有价值——你蛋白质组学的领域判断 + 工程 review 能力，加上 Claude Code 的代码生产力，才是 $1+1>2$ 。

3. 简历叙事的风险——如果你 100% 让 Claude Code 写完所有代码，面试官追问"为什么 router 这里用 LLM 而不是规则"、"为什么 RRF 的 $k=60$ "，你答不上来就会很尴尬。所有架构决策必须是你自己做的，Claude Code 只负责实现。

建议的使用边界

我建议你这样划分工作：

任务类型	用什么
架构决策（用什么 embedding、表格走 SQL 还是序列化、评估指标选什么）	你 + Claude.ai 讨论（像现在这样）
写代码（实现 <code>dense_index.py</code> 、 <code>hybrid.py</code> 这类已经设计好的模块）	Claude Code
调试（pip 报错、import 失败、Streamlit 起不来）	Claude Code
跑测试和评估（pytest、跑评估集、生成对比表）	Claude Code
README、demo gif、git 提交策略	你 + Claude.ai
review 每一段生成的代码	必须你亲自做
评估集标注	必须你亲自做（这是你的领域价值）

一个具体的建议 workflow

把上一轮的 5 个 Phase 跟工具映射一下：

Phase 1（脚手架）——纯 Claude Code 最快。一句"基于这个目录结构和 requirements 创建所有空文件和 `__init__.py`"就搞定。

Phase 2（垂直切片）——混合。架构和 prompt 跟 Claude.ai 讨论清楚，具体文件让 Claude Code 写，但你逐文件 review，每个文件至少自己读懂、能口头解释。

Phase 3（hybrid + agent）——Claude Code 主导写，你重点 review 三个文件：hybrid.py（RRF 的逻辑你要会讲）、router.py（这是项目亮点）、pipeline.py（整体编排）。

Phase 4（评估）——评估集标注必须你自己来，跑评估脚本和生成对比表用 Claude Code。

Phase 5（发布）——README 你主导写（这是面试官读你的"作品介绍"），Streamlit 部署和 CI 配置交给 Claude Code。

怎么开始用

如果你决定上手，最简路径：

1. 在 WSL2 终端跑 `npm install -g @anthropic-ai/claude-code`（需要 Node.js，没装的话先 `sudo apt install nodejs npm`）
2. 跑 `claude` 启动，第一次会让你登录 Anthropic 账号
3. `cd ~/projects/proteomics-rag` 进入项目目录
4. 启动 Claude Code，第一句话直接给它你之前那份目录结构 + requirements，让它生成 Phase 1 的脚手架

具体的入门细节、目前最新的命令、跟 VS Code 集成的方式，你可以查官方文档——这些经常更新，我手里的版本可能不是最新。

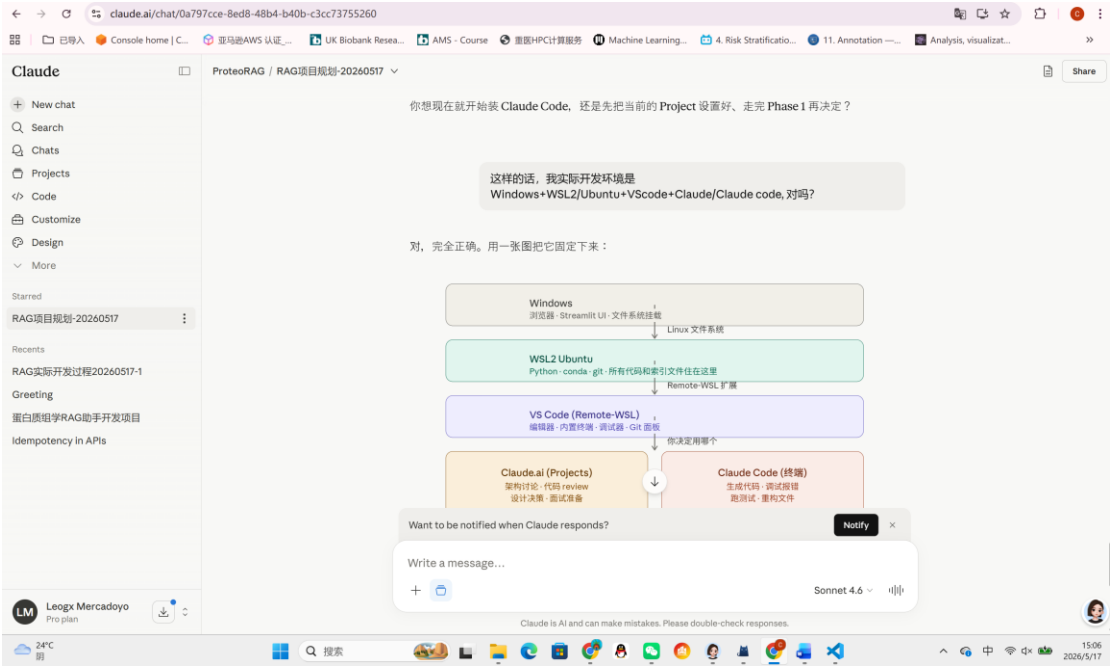
一句话总结

用，但要把自己定位成 **"AI-assisted engineer"**，不是 **"vibe coder"**。你的核心价值——蛋白质组学领域判断 + 工程架构决策 + 评估集标注——必须在你手里。Claude Code 帮你把"想清楚的东西高速实现出来"，省下的时间用来打磨真正的差异化（评估实验、README、面试讲稿）。

7. 开发环境总结

Prompt:

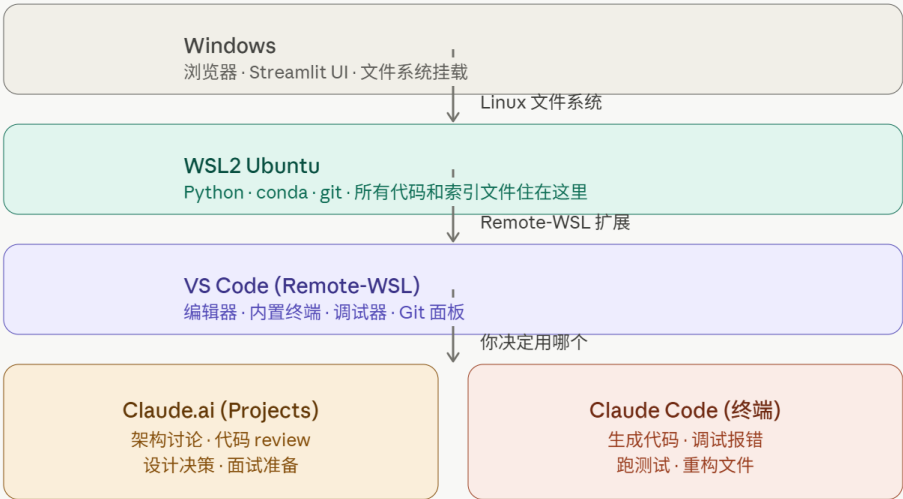
这样的话，我实际开发环境是 Windows+WSL2/Ubuntu+VScode+Claude/Claude code, 对吗？



Response:

这样的话，我实际开发环境是
Windows+WSL2/Ubuntu+VScode+Claude/Claude code, 对吗？

对，完全正确。用一张图把它固定下来：



对，完全正确。用一张图把它固定下来：对，完全正确。用一句话固定认知：

Windows 是显示器，WSL2 Ubuntu 是真正的计算机，VS Code 是你的操作台，Claude.ai + Claude Code 是你的 AI 结对伙伴。

补充两个容易混淆的细节：

Claude.ai 和 Claude Code 不是替代关系，是分工关系。 你现在用的 Claude.ai（浏览器里）负责"想清楚"——架构讨论、权衡取舍、面试叙事。 Claude Code（WSL2 终端里）负责"做出来"——生成文件、调 bug、跑脚本。 两个同时开着，相互配合。

Claude Code 运行在 WSL2 里，不是 Windows 里。 安装时在 WSL Ubuntu 终端里跑命令，不是在 Windows PowerShell 里。这样它才能直接读写你的项目文件，也就是 `~/projects/proteomics-rag/` 下面的所有内容。

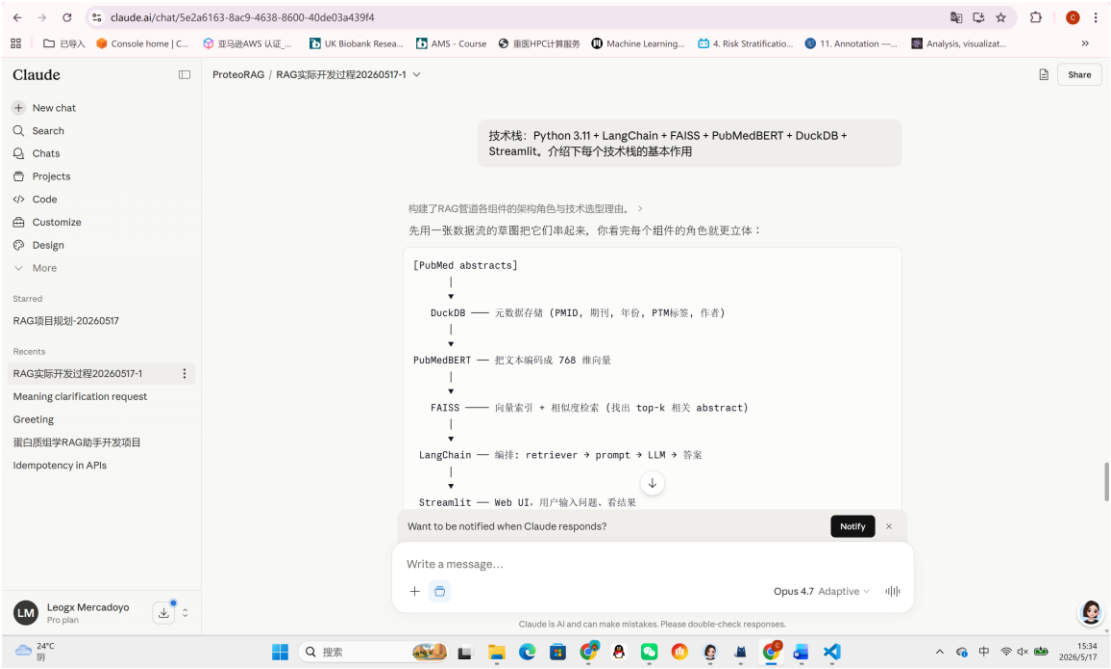
这个环境组合确认好之后，下一步就可以动手了。你现在可以：

1. 把这个对话加入 Claude Project (截图里那个 "Add to project"), 让后续每个新对话都自动带上项目背景
2. 在 WSL 终端里装 Claude Code (`npm install -g @anthropic-ai/claude-code`)
3. 喊我开始 Phase 1 脚手架

8. 技术栈总结

Prompt:

技术栈：Python 3.11 + LangChain + FAISS + PubMedBERT + DuckDB + Streamlit。介绍下每个技术栈的基本作用



Response:

构建了RAG管道各组件的架构角色与技术选型理由。 >

先用一张数据流的草图把它们串起来，你看完每个组件的角色就更立体：



先用一张数据流的草图把它们串起来，你看完每个组件的角色就更立体：

[PubMed abstracts]

|

▼

DuckDB —— 元数据存储 (PMID, 期刊, 年份, PTM 标签, 作者)

|

▼

PubMedBERT —— 把文本编码成 768 维向量

|

▼

FAISS —— 向量索引 + 相似度检索 (找出 top-k 相关 abstract)

|



LangChain — 编排: retriever → prompt → LLM → 答案

|



Streamlit — Web UI, 用户输入问题、看结果

Python 3.11——基础语言。

选 3.11 而不是 3.12/3.13 的理由: 3.11 比 3.10 快 10-60% (CPython 的 specializing adaptive interpreter), 错误提示极大改善 (错误指到具体表达式而不是整行); 同时所有主流 ML/科学计算库 (PyTorch、faiss、transformers、langchain) 都对 3.11 有完整 wheel 支持, 3.12 偶尔还有 C 扩展未编译的尴尬。是"够新但够稳"的甜蜜点。面试讲: **选最新稳定版而不是 latest**, 是工程取舍意识。

LangChain——RAG 链路的编排框架, 相当于"乐高积木"。

它本身不做检索、不做嵌入、也不是 LLM, 它的价值是**把这些组件用统一接口串起来**: Retriever | PromptTemplate | LLM | OutputParser 这种管道写法。还提供现成的 PubMed loader、文档分块器、prompt 模板、引用追踪等。

替代品和取舍:

- **LlamaIndex**: 更专注 retrieval/indexing, 抽象更"重", 对新手反而难调试
- **手写裸代码**: 完全可行, 控制力最强, 但你要自己处理 prompt 模板、上下文拼接、引用回溯——这些 LangChain 都包好了
- **选 LangChain 的额外理由**: 招聘市场认知度最高, HR 关键词扫描友好

注意一个反共识 talking point: 很多人骂 LangChain 抽象层太多。你可以在面试时主动说"我只用 LangChain 的核心组件——LCEL、Retriever 接口、PromptTemplate——避开它的 Agent 和复杂 Chain, 因为那些抽象成本高于收益"。这立刻把你和"调包侠"区分开。

FAISS——向量数据库 / 相似度搜索引擎, Facebook AI Research 出品。

作用: 你把 10 万条 abstract 用 PubMedBERT 编码成 10 万个 768 维向量后, 需要一个东西回答"和这个 query 最相似的 top-10 是哪些"。暴力遍历 10 万次余弦相似度太慢, FAISS 用 IVF、HNSW、PQ 等近似最近邻算法把检索压到毫秒级。

替代品和取舍:

- **Chroma:** API 友好, 自带持久化, 但底层其实也是 hnswlib, 性能和灵活度不如 FAISS
- **Pinecone / Weaviate:** 云端托管, 但要钱、要联网、对求职 demo 不必要
- **pgvector:** 如果你已经有 Postgres, 可选; 从零起手反而重
- **选 FAISS 的理由:** 工业级 (Meta 内部就用), 完全本地、免费, CPU 版本对 <100 万条 abstract 完全够用, 且**控制粒度最细**——你能自己选 index type, 这是面试可讲的细节深度

PubMedBERT——领域专用嵌入模型, 这是你项目的**领域护城河**。

作用: 把"S-nitrosylation of GAPDH at Cys152 inhibits glycolysis"这种句子编码成 768 维向量, 相似含义的句子向量距离近。

为什么不用通用模型 (OpenAI text-embedding-3-small、sentence-transformers/all-MiniLM-L6-v2):

- 通用模型词表里 "S-nitrosylation"、"PRMT5"、"asymmetric dimethylarginine" 很可能被切成多个子词, 语义被稀释

- PubMedBERT 在 PubMed 全文 + abstract 上从头预训练（不是 fine-tune），词表本身就是生物医学语料统计出来的，"GAPDH"、"CysNO"、"ADMA" 这类术语是 single token
- **这是你的故事点：**面试讲"我做了一个对照实验，用 PubMedBERT vs all-MiniLM 在 50 条 PTM query 上比 Recall@10，PubMedBERT 高 X%"。后面 Eval harness 那步会做这个。

类似选择：BioBERT、SciBERT、SapBERT。PubMedBERT 在 BLURB benchmark 上整体最强，且 Hugging Face Hub 上下载方便（microsoft/BiomedNLP-PubMedBERT-base-uncased-abstract）。

DuckDB——嵌入式分析型数据库，存元数据和做 SQL 过滤。

作用：FAISS 只管"向量相似"，但你经常需要**结构化过滤**——"S-亚硝基化相关的 abstract，且 2020 年后发表，且发在 IF>10 的期刊"。这种查询用 SQL 是最自然的。DuckDB 让你直接 SELECT pmid, title FROM papers WHERE ptm_type='S-nitrosylation' AND year>=2020。

替代品和取舍：

- **SQLite：**也嵌入式，但是行存储，做聚合分析慢一个量级
- **Postgres：**要起服务，对单人项目过重
- **pandas + parquet：**可以，但每次过滤都要 load 整个表，内存吃紧
- **选 DuckDB 的理由：**列存储（OLAP 友好）、零配置（单文件就是数据库）、和 pandas/polars/Arrow 零拷贝互操作、SQL 标准。**这是 2023-2026 数据工程圈的明显趋势**，写在简历上能引发面试官话题。

架构上 DuckDB 和 FAISS 是**互补**关系：DuckDB 管"是什么"（元数据），FAISS 管"像什么"（语义）。一次查询通常先 FAISS 拿 top-100 候选 PMID，再用 DuckDB 按时间/期刊/PTM 类型过滤到 top-10。这个 hybrid retrieval 设计本身就是亮点。

Streamlit——Python 写 Web UI，给项目一个"门面"。

作用：用户输入"PRMT5 在乳腺癌中的底物有哪些？"，看到答案+引用的 abstract+可点击的 PMID 链接。没有 UI 的项目在 GitHub 上视觉冲击力差一大截。

替代品和取舍：

- **Gradio**：更偏 ML demo（拖拽图片、滑动条），不适合带表格、引用、多 tab 的应用
- **FastAPI + React**：专业，但你一个人做求职 demo 不值得投入两周写前端
- **选 Streamlit 的理由**：纯 Python、几十行代码出像样界面、可一键部署到 Streamlit Community Cloud（免费），GitHub README 里能直接放 live demo 链接，面试时点开就能演示

整体设计哲学（面试 elevator pitch）

这个项目我用了一套"轻量但严肃"的栈：FAISS 做向量、DuckDB 做元数据、PubMedBERT 做领域嵌入，LangChain 只用核心管道、Streamlit 做演示。没用任何云服务、任何付费 API、任何重型框架——目的是证明 RAG 系统的核心价值在于**检索质量和领域适配**，而不是堆基础设施。所有组件都是本地的、可复现的、可被同行 clone 后一键跑通的。

这一段你可以反复打磨，面试时一定会用到。

要不要先继续走第 2 步（uv 装环境），还是想先看一下整体架构图（我可以画一张更细的，标出每个 Python 模块对应哪个组件）？

(五) 实际开发记录-20260517

1. 第 1 步：建项目骨架

Prompt: 总体线路

下面请指导我一步一步完成开发过程，我启动了 VScode，如何所示？接下来从哪里入手？

Response:

看你的状态：WSL: Ubuntu 已连接、base conda 环境激活、还没打开任何文件夹。位置对了，可以开始。

先给你一个整体路线图，让你心里有谱（后面每一步我们再展开）：

1. 项目骨架 + Git 初始化（今天这一步）
2. Python 环境（uv + Python 3.11，理由下面讲）
3. 开发工具链（ruff + pytest + pre-commit，第一天就上）
4. README-driven 设计（先写故事再写代码，逼自己想清楚 scope）
5. 数据层：PubMed/UniProt 抓取 → DuckDB
6. 检索层：PubMedBERT 向量化 → FAISS
7. RAG chain: LangChain orchestration
8. UI: Streamlit
9. Eval harness（这是你和"调包侠 RAG 项目"拉开差距的关键）
10. 部署 + GitHub 仓库门面

第 1 步：建项目骨架

1.1 决定项目放在哪（这一步很关键）

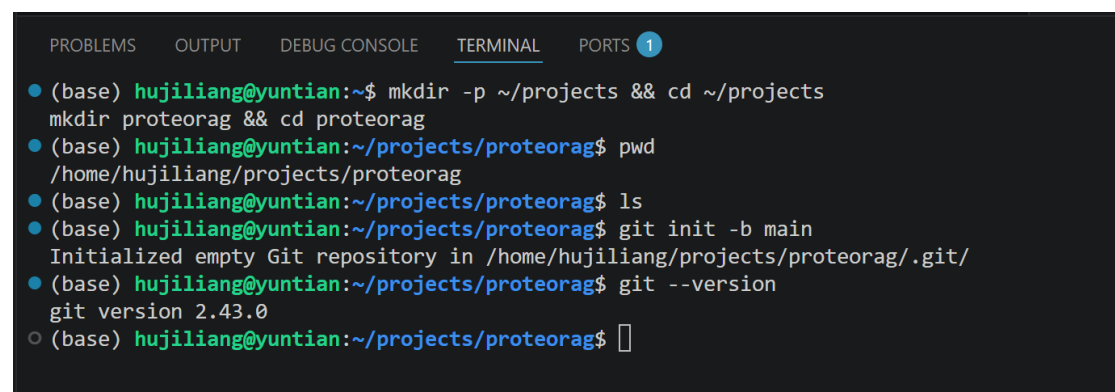
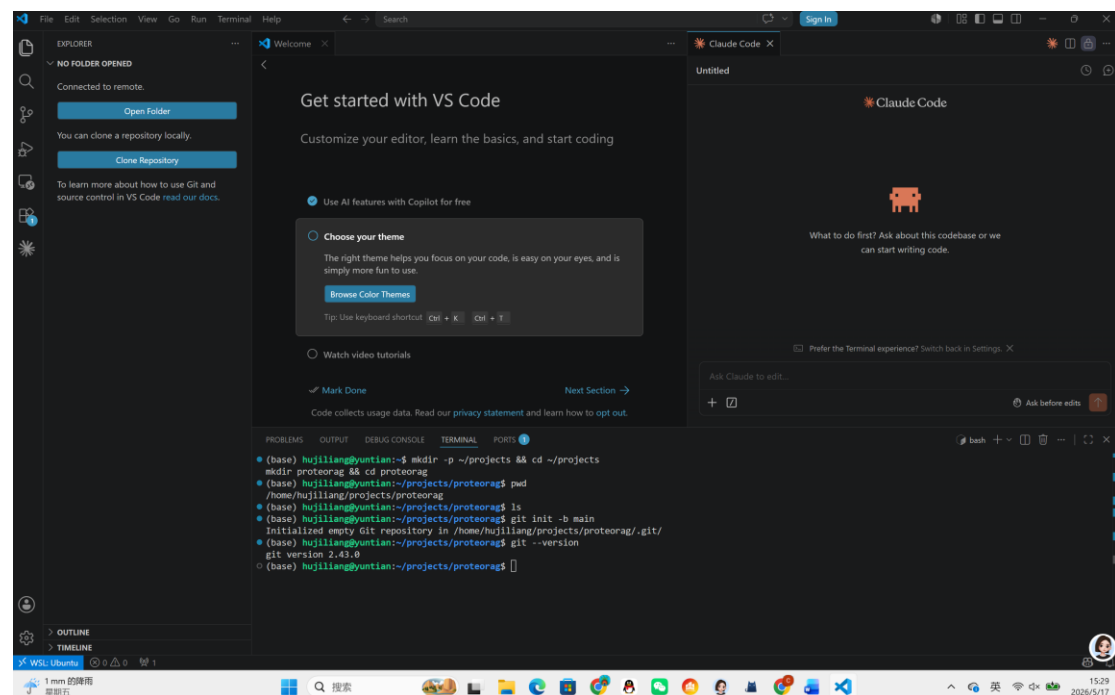
WSL2 Linux 文件系统内 (~/projects/), 不要放在 /mnt/c/Users/... 下。

理由 (面试可以讲): WSL2 跨文件系统 IO 有 10-20 倍性能损失, git status、pip install、向量索引读写都会肉眼可见地慢。一个嵌入十万条 abstract 的项目, 差别可能是 2 分钟 vs 30 分钟。

在 VS Code 底部的 Terminal 里跑:

```
mkdir -p ~/projects && cd ~/projects
```

```
mkdir proteomics-rag && cd proteomics-rag
```



1.2 初始化 Git (先于一切)

`git init -b main`

用 `-b main` 直接设默认分支名，避免后面 GitHub 默认是 `main` 但本地是 `master` 的尴尬。

```
git init -b main
```

这条命令的意思是：在当前目录初始化一个新的 Git 仓库，并将默认分支名设为 `main`。

拆开来：

- `git init` — 在当前文件夹创建一个新的 Git 仓库（生成隐藏的 `.git` 文件夹）
- `-b main` — 指定初始分支名为 `main`

为什么要加 `-b main` ？

Git 历史上默认分支叫 `master`，但 GitHub 现在默认用 `main`。

如果你本地用 `git init`（不加 `-b main`），分支名是 `master`，推到 GitHub 时就会出现分支名不一致的麻烦。

加了 `-b main` 之后，本地和 GitHub 都是 `main`，后续操作更顺畅。

为什么要初始化 Git 仓库？

简单说：Git 需要先"认识"你的项目文件夹，才能开始帮你追踪和管理代码。

打个比方

想象你有一个普通的文件夹，里面放着你的代码文件。

这个文件夹现在就是个"普通文件夹"，Git 完全不知道它的存在。

运行 `git init` 之后，Git 会在里面创建一个隐藏的 `.git` 文件夹，从此开始"监视"这个项目——记录你每次改了什么、什么时候改的、谁改的。

初始化之后你能做什么？

- **版本回退** — 写坏了？一键回到之前能用的版本
- **查看历史** — 看清楚每次改动了什么
- **多人协作** — 和别人一起开发，不会互相覆盖
- **推送到 GitHub** — 把代码备份/分享到云端

什么时候需要 `git init` ？

只有**新建项目**的时候才需要运行一次。

如果是从 GitHub 上下载别人的项目 (`git clone`)，则不需要，因为 `.git` 已经包含在里面了。

1.3 创建目录结构

这里要做一个选择：**src layout** vs **flat layout**。我推荐 **src layout**：

```
mkdir -p src/proteomics_rag/{data,embedding,retrieval,chain,ui}
```

```
mkdir -p tests notebooks data/{raw,processed,index} configs scripts docs
```

```
touch src/proteomics_rag/__init__.py
```

```
touch src/proteomics_rag/{data,embedding,retrieval,chain,ui}/__init__.py
```

为什么选 **src layout** (面试 talking point)：

- 强制项目以"包"方式被安装 (`pip install -e .`)，而不是靠当前目录的隐式导入路径
- 测试时如果忘了安装包就会 `import error`，能立刻暴露环境配置错误 (flat layout 会"恰好能用"，掩盖问题)
- 这是 PyPA、Hypermodern Python 这套主流工具链的标准实践，看仓库的人一眼就知道你不是第一次写 Python 项目

子模块的拆分对应未来 RAG 的数据流：data (抓取/清洗) → embedding (向量化) → retrieval (FAISS 检索) → chain (LangChain 组合) → ui (Streamlit)。每一层都能独立测试。

1.4 关键文件：.gitignore、README.md、LICENSE

.gitignore（直接复制以下，覆盖了 Python + 数据 + IDE + OS）：

```
cat > .gitignore << 'EOF'
```

```
# Python
```

```
__pycache__/
```

```
*.py[cod]
```

```
*.egg-info/
```

```
.pytest_cache/
```

```
.ruff_cache/
```

```
.mypy_cache/
```

```
.coverage
```

```
htmlcov/
```

```
dist/
```

```
build/
```

```
# Environments
```

```
.venv/
```

```
.env
```

```
.env.local
```

```
# Data & models (track via DVC or release artifacts, not git)
```

```
data/raw/
```

data/processed/

data/index/

*.faiss

*.duckdb

*.duckdb.wal

models/

Jupyter

.ipynb_checkpoints/

notebooks/*.ipynb_checkpoints

IDE / OS

.vscode/

.idea/

.DS_Store

Thumbs.db

Logs

*.log

logs/

EOF

为什么 **data/** 整个忽略：原始 PubMed abstract、FAISS 索引、DuckDB 文件可能是 GB 级别，git 不适合存。后面我们用 release artifact 或 HuggingFace Hub 分发预计算的索引，让别人 clone 后跑个 make download-index 就能用。这也是工程严谨度的体现。

README.md 先放占位（之后第 4 步会做 README-driven design）：

```
cat > README.md << 'EOF'
```

```
# proteomics-rag
```

A domain-specialized retrieval-augmented generation system for proteomics literature,

with a focus on post-translational modifications (PTMs) including S-nitrosylation and arginine methylation.

****Status****: 🚧 Under active development.

```
## Why this project
```

(TODO: motivation, what existing tools miss, what this adds)

```
## Architecture
```

(TODO: diagram)

Quickstart

(TODO)

EOF

LICENSE——选 MIT（求职场景最常见、最无歧义）：

```
curl -sL https://raw.githubusercontent.com/licenses/license-templates/master/templates/mit.txt \
```

```
| sed "s/{{ year }}/${(date +%Y)}/; s/{{ organization }}/Hu Jiliang/" > LICENSE
```

把 Hu Jiliang 换成你想署的名字。

1.5 在 VS Code 里打开这个文件夹

回到 VS Code，Ctrl+Shift+P → File: Open Folder → 选
/home/hujiliang/projects/teomics-rag。

或者更快，在终端里：

```
code .
```

VS Code 会重新加载并把这个文件夹设为工作区。

1.6 第一次提交

```
git add .
```

```
git commit -m "chore: initial project skeleton"
```

Checkpoint: 完成 1 后告诉我

跑完以上，给我看一下 `ls -la` 和 `tree -L 2 -a -I '.git'` 的输出（如果没装 `tree`：
`sudo apt install tree`）。确认骨架对了，我们进 **第 2 步：uv 环境 + Python**

3.11。

在那一步我会跟你讲清楚 为什么选 **uv** 而不是 **conda** (你做 bio 的同事大概率会问这个), 以及怎么把 PubMedBERT、faiss-cpu、torch 这些"传统上 conda 才装得顺"的包用 uv 装好。